



Fast and scalable sliding-window-based algorithms for mining frequent weighted utility patterns over dynamic quantitative data streams

Nguyen Le ^{a, *}, Ham Nguyen ^{b, *}, Minh Nguyen ^c, Huong Bui ^a, Bay Vo ^a,
Unil Yun ^d

^a Faculty of Information Technology, HUTECH University, Ho Chi Minh City, Viet Nam

^b Faculty of Information Science, Saigon University, Ho Chi Minh City, Viet Nam

^c Faculty of Mathematics, Mechanics, and Informatics, VNU University of Science, Hanoi, Viet Nam

^d Department of Computer Engineering, Sejong University, Seoul, the Republic of Korea

ARTICLE INFO

Keywords:

Dynamic data streams
Frequent weighted utility pattern
Sliding-window model

ABSTRACT

Frequent weighted utility pattern (FWUP) mining represents a significant extension within the domain of frequent pattern mining that aims to identify frequently occurring and significant patterns in quantitative databases. In today's world, streams of data are constantly being generated rapidly, requiring appropriate mining methods to swiftly discover useful information from them. Previous algorithms for mining FWUPs were conceived for databases characterized by a predetermined quantity of transactions and are not amenable to direct application on dynamic data streams. Therefore, this study introduces the problem of mining FWUPs over dynamic quantitative data streams based on the sliding-window model. We propose two algorithms, named FWUDS-CT and FWUDS-DWT, to address this problem efficiently by reusing knowledge from previous windows without reprocessing the entire data. Furthermore, we also develop methods to perform real-time data updates. Empirical results show that our proposed method achieves superior performance in execution time and scalability compared to prior approaches for mining FWUPs over data streams.

1. Introduction

The frequent pattern (FP) mining challenge was initially presented by Agrawal and colleagues [1], becoming an essential problem in data mining. The task of FP mining is to identify all patterns whose occurrence frequency meets or exceeds a user-specified threshold on a binary transaction database. However, FP mining has a limitation when applied to real applications, which is that it only focuses on the frequency and occurrence of patterns and ignores other important information. Consequently, the problem of pattern mining has expanded in various directions, such as high utility patterns (HUPs) [2,3], high average utility patterns (HAUPs) [4], sequential patterns [5],... A recent research direction of interest is mining frequent weighted utility patterns (FWUPs). FWUP mining focuses on both the frequency and the significance of patterns in quantitative databases. Various approaches have been developed to address this

* Corresponding author.

E-mail addresses: lhb.nguyen@hutech.edu.vn (N. Le), ndham@sgu.edu.vn (H. Nguyen), minhnhth@hus.edu.vn (M. Nguyen), bd.huong@hutech.edu.vn (H. Bui), vd.bay@hutech.edu.vn (B. Vo), yunei@sejong.ac.kr (U. Yun).

<https://doi.org/10.1016/j.ins.2026.123488>

Received 10 October 2024; Received in revised form 6 April 2026; Accepted 7 April 2026

Available online 8 April 2026

0020-0255/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

problem, including MBiS [6], WUN-Miner [7], FWUP-SWUNL [8] for static databases, and dFWUT [9] and dFWUNL [9] for dynamic databases.

In recent years, the rapid advancement of information technology has led to a massive increase in the volume of data. Humans or machines often generate these data in the form of data streams, and they are collected using sales data, social networks, and system logs. Extracting useful information from data streams under time and memory constraints has become a need and challenge in data mining. Mining FWUP in dynamic data streams is crucial for large-scale data processing systems, where information is perpetually evolving and necessitates immediate analysis of both occurrence frequency and the value contribution of data elements. Within the e-commerce sector, FWUP facilitates the identification of product combinations that are frequently purchased together and yield substantial profits, thereby aiding in the development of dynamic recommender systems, enhancing personalized promotional strategies, and enabling real-time inventory management. In the field of social media analysis, FWUP contributes to the recognition of topics, content, or hashtags that not only have a notable degree of popularity but also possess significant influence, thereby effectively meeting the aims of trend identification, brand health evaluation, and misinformation detection. Furthermore, in the financial and banking industry, FWUP plays a vital role in monitoring and identifying prevalent transaction patterns associated with high monetary value or elevated risk levels, thereby enhancing the efficacy of fraud detection and risk management systems. The ability to mine FWUP swiftly in real-time and adapt to fluctuations in data and weights is a challenge that traditional pattern mining methods struggle to address.

Applying traditional FWUP mining methods on data streams is not suitable due to the data's nature and limitations. It is also due to the inability to use past computational results to optimize the mining process. Furthermore, real-world applications generate extensive amounts of data quickly, requiring the mining process to be performed at high speed in a single database scan. The sliding window model is frequently employed for extracting patterns from continuous data streams [4,10,11]. This model includes a window with a fixed number of the most recent data panes. As a new pane is added, the oldest pane is removed. Therefore, the sliding window model always ensures that the newest data is mined since often recent data provides the most value. Furthermore, pattern mining based on this sliding window approach requires a smaller memory footprint compared to other methods because it does not have to process and store the entire database. Several algorithms have been proposed for discovering HUPs [10,12], erasable patterns (EPs) [13], and other relevant constructs within data streams employing the sliding window methodology. Yet, it is currently needed to apply this to FWUP mining.

With the above motivations, this paper introduces the problem of mining FWUPs over dynamic quantitative data streams using the sliding window model, where item weights may change. Our main goal is to design efficient, single-pass algorithms that can effectively manage and mine these patterns, accounting for time-varying item weights, without needing to rescan the entire data window.

To achieve this objective, the primary contributions of this work are outlined below: (1) This study provides a formal conceptualization and framework for mining FWUPs over dynamic quantitative data streams, grounded in the sliding window paradigm. (2) We propose two new data structures, CTset (Circular Tidset) and DSWUN-tree (Dynamic Streaming Weighted Utility Node tree), for efficient data storage and processing for data streams managed under the sliding window model. The distinctive feature of CTset lies in its circular tidset management mechanism, which enables the reuse of the transaction identifier domain while preserving the logical uniqueness of transactions within the sliding window. This design effectively prevents numerical overflow and reduces storage overhead. Meanwhile, the DSWUN-tree structure provides a compact representation of all information within the sliding window and supports dynamic updates as new data arrive, eliminating the need for database rescans. From this, we can leverage the superiority of the WUN-list data structure to speed up the mining process. (3) We develop two mining algorithms for FWUPs over data streams using the sliding window model, named FWUDS-CT (Frequent Weighted Utility pattern mining over Data Streams using Circular Tidset) and FWUDS-DWT (Frequent Weighted Utility pattern mining over Data Streams using DSWUN-tree), which utilize the data structures mentioned above. (4) We conduct experiments to compare these two methods' runtime and memory usage on various datasets. The empirical findings indicate that both algorithms ensure reliability and correctness, with the FWUDS-DWT algorithm working more efficiently than FWUDS-CT.

Here is how the paper is structured. [Section 2](#) reviews related work. [Section 3](#) formulates the problem of mining FWUPs using a sliding window model on dynamic quantitative data streams. The section also covers basic concepts and knowledge of mining FWUPs. Next, the two algorithms FWUDS-CT and FWUDS-DWT for mining FWUPs over data streams are presented and described in detail in [sections 4 and 5](#). Empirical evaluations and findings are reported in [Section 6](#), and [section 7](#) provides the complexity analysis and in-depth discussion about the proposed methods. Finally, the conclusion and prospective research avenues are discussed in [Section 8](#).

2. Related work

2.1. Mining patterns on quantitative databases

Quantitative databases frequently emerge in retail industry databases. In a quantitative database, each transaction records the quantity of products purchased, and each product has a varying weight. This weight could be the price, profit, or the importance of the product within the database. Two methodologies have been advanced to extract significant insights from quantitative databases. They are high utility pattern mining and frequent weighted utility pattern mining.

2.1.1. Mining high-utility patterns

High utility pattern mining involves identifying all patterns with utility values that meet or exceed a predetermined minimum utility threshold specified by the user. This approach addresses the limitation of traditional FP mining, which is ignoring items that

bring great benefits but have low occurrence frequency. In HUP mining, the main challenge arises because the search space is typically combinatorically large when the anti-monotonicity property does not work, as the utility values generally increase with pattern length. To solve this problem, Liu et al. [14] introduced the Two-Phase algorithm. It uses the principle of Transaction-weighted Utilization as an upper bound satisfying the downward closure property. The Two-Phase algorithm is fundamentally derived from the Apriori methodology, so it has the limitation of scanning the database many times and generating many candidates. Next, the UP-Growth [15] and UP-Growth+ [16], inspired by FP-Growth, incorporate several pruning strategies to improve computational efficiency during mining. Later, Liu and Qu [17] proposed the HUI-Miner algorithm, which employs a novel utility list data structure to mine HUPs. HUI-Miner only needs a single database scan to gather and store necessary information in the utility list. This structure enables rapid calculation of utility values. The FHM algorithm [18] proposed by Fournier-Viger et al. uses Estimated Utility Co-occurrence Pruning to quickly remove unpromising itemsets, thereby reducing the number of join operations. Next, Zida et al. [19] developed the EFIM algorithm and introduced two upper bounds: local utility and sub-tree utility. EFIM uses database projection and transaction merging techniques to minimize scanning overhead and improve HUP mining efficiency. Recently, researchers proposed many new HUP mining algorithms focusing on optimizing the utility-list-based representations. The HMiner algorithm [20] uses the new compact utility list data structure and the virtual hyperlink data structure. The HUI-Miner* algorithm employs the utility-list* structure [21]. The FOTH algorithm utilizes the novel IndexSet structure [22]. These algorithms demonstrated their efficiency in mining time and memory resources when applied to both dense and sparse datasets. Furthermore, researchers have extended HUP mining to variety other data mining tasks, such as HAUP mining [4], high utility sequential pattern mining (HUSP) [23], and skyline pattern mining [24].

2.1.2. Mining FWUPs

In the retail industry, FP mining delivers information about frequently purchased products, while HUP mining provides insights into the most profitable products. However, in practice, companies also aim to identify products that both bring high value and sell well to bolster their business strategies. To tackle this challenge, Khan et al. introduced a novel research direction known as weighted utility association rules mining [25]. The proposed approach comprises two phases: the first phase finds FWUPs, and the second phase generates rules from these patterns. To accomplish this, the authors proposed two new metrics: transaction weighted utility (*twu*) along with weighted utility support (*wus*). These metrics integrate both frequency and significance and notably utilize the downward closure property, which simplifies the process of pruning the search space. Subsequently, Vo et al. [26] introduced the MWIT-tree structure, which employs a tidset structure and achieves the mining of FWUPs with just a single database scan. Next, Nguyen et al. [6] developed the MBIS structure to enhance tidset storage optimization and improve the efficiency of mining. The MBIS structure stores information of consecutive sub-segments in tidsets to optimize memory usage, and the authors also introduced an efficient method for constructing new MBIS structures. Following that, Bui et al. [7] presented the WUN-set structure and its corresponding WUN-Miner algorithm. More recently, Nguyen et al. [8] introduced the FWUP-SWUNL algorithm based on the SWUN-list structure, and proposed several theorems to accelerate the mining process. Both the WUN-set and SWUN-list originate from the WUN-tree, however, the SWUN-list provides superior data compression, thereby reducing both execution time and memory footprint. Experimental findings suggest that the FWUP-SWUNL algorithm is currently the most efficient algorithm for mining FWUPs. Additionally, Tran et al. [27] utilized FWUP for text clustering, achieving higher accuracy than methods based on frequent patterns. Meanwhile, Nguyen et al. [28] developed the MINE FWUIS and FAST MINE FWUIS algorithms, designed for mining FWUPs in hierarchical quantitative databases that organize items in a hierarchical structure, indicating interrelated item categories through a generalization relationship.

The aforementioned studies on FWUP mining have concentrated on static quantitative databases, in which the weights of items are constant. However, in practical scenarios, the significance of each product can fluctuate due to various factors, including discount strategies, item liquidation, and shifts in consumer behavior. Consequently, it is necessary to adjust the weight of each product appropriately. Nguyen et al. [9] addressed this challenge by mining FWUPs from dynamic quantitative databases (dQDB), where item weights may vary over time. Such dQDB are commonly encountered in practical applications. The weight of each data item could signify profit, price, or other benefits of products, and these values can change with each transaction. For instance, an item's price may alter depending on the quantity bought (such as receiving a discount for bulk purchases) or vary seasonally (like different pricing for cooling appliances in summer versus winter). This leads to the highly practical problem of mining FPs from dQDB with changing weights. The authors proposed two algorithms based on baseline structures: the dFWUT algorithm, which uses a tidset structure, and the dFWUNL algorithm, which utilizes the WUNList structure, an extension of the N-List structure [29]. Experimental evaluations on both sparse and dense datasets have demonstrated that the dFWUNL algorithm is more efficient than the dFWUT algorithm.

2.2. Mining patterns over data streams

Data streams are characterized by their potentially infinite length, massive and continuous flow, unbounded nature, temporal ordering, and fast-changing features. Traditional pattern mining methods are not feasible for data streams because storing and processing the entire data stream requires substantial storage and computational capacity. Numerous studies have been conducted on pattern mining in data streams, concentrating on three primary models: the landmark model [30,31], the damped window model [32,33] and the sliding window model [4,13,34]. Each model has its own advantages and disadvantages, and selecting a mining model depends on the inherent attributes of the data as well as the specific needs of the user. In the landmark model, data from a specific starting point, known as a landmark, to the current time is considered (Fig. 1(a)). This landmark can be fixed, changeable, or resettable throughout the mining process. A key characteristic of this model is that all transactions within this time frame are equally important, rendering it appropriate for contexts in which both new and old data hold equal significance. Unlike the landmark model, the damped

window model weights transactions according to their chronological order in the data stream (Fig. 1(b)). In this model, older transactions diminish in impact; therefore, their significance and weight decrease with each new data addition. Users define a decay factor, which ranges from 0 to 1, to dictate this decrease. Each time a new transaction is added, the weights of previous transactions are multiplicatively reduced by this decay factor over time. While the damped window model emphasizes the differences between transactions, it demands considerable resources to maintain and manage the entire dataset. The sliding window mechanism is another popular approach in data stream pattern mining. In this model, the data stream is segmented into equal-sized data panes. A window, which is fixed in size and contains multiple panes retains only the most contemporaneous data. As new data panes enter, the oldest pane is discarded to keep the window size constant (Fig. 1(c)). Pattern mining is then conducted on this refreshed window. The sliding window model facilitates quick extraction of useful information from the latest data, although selecting the optimal pane and window sizes poses challenges for users.

Pattern mining algorithms for data streams have been extensively researched and applied to various types, including FPs [35,36], HUPs [10,33,37], EPs [13,32], among others, achieving notable success. Generally, these algorithms focus on solving the following main problems. Firstly, the algorithms are designed to minimize data scanning to just one pass, given the large volume and rapid influx of data streams. Secondly, new data structures have been developed to efficiently store, manage, update, and discover patterns when new data arrives. Thirdly, the algorithms are crafted to ensure reliability, accuracy, and efficient use of resources. However, as of now, there is no existing research on mining FWUPs in data streams.

A critical analysis of prior literature, summarized in Table 1, reveals several significant research gaps that motivate our proposed approach. Most existing high-performance FWUP mining algorithms [7,8] are fundamentally designed for static databases. Their reliance on full data access and the assumption of fixed datasets render them unsuitable for the transient, high-velocity nature of data streams. Furthermore, even the most relevant state-of-the-art (SOTA) dynamic algorithm, dFWUNL [9], which is highly effective for quantitative databases, wherein the weights assigned to items may vary, suffers from a major inefficiency. It was not designed for the sliding window model and requires two complete scans of the current window. This repeated re-scanning is computationally prohibitive for real-time, single-pass stream processing. Concurrently, while the field of stream mining has produced numerous pattern mining algorithms, these methods do not address our specific problem. They are not designed to handle the combined challenges of FWUPs, quantitative databases, and dynamically varying item weights. Therefore, a clear research gap exists for an efficient framework that can reuse previously computed information, rapidly update data in real time as the window slides, and extract all FWUPs from dynamic quantitative data streams, precisely the challenge that our proposed methods aim to address.

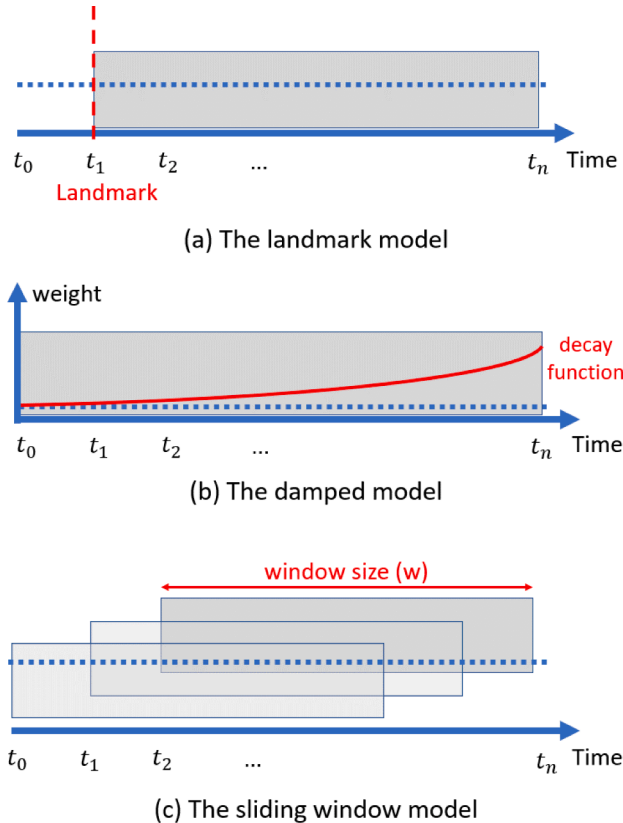


Fig. 1. The types of models for pattern mining over data streams.

Table 1

An examination of algorithms pertaining to pattern mining.

Algorithm (Year)	Author	Data structure	Patterns	Databases
Apriori (1994)	Agrawal et al.	None	FPS	Binary transactions (BT)
Eclat (1997)	Zaki et al.	Tidset	FPS	BT
FP-Growth (2000)	Han et al.	FP-tree	FPS	BT
Two-Phase (2005)	Liu et al.	None	HUPS	Quantitative transactions (QT)
MFI-TransSW (2009)	Li et al.	Bit sequence	FPS	Binary data streams
UP-Growth (2010)	Tseng et al.	UP-tree	HUPS	QT
PrePost (2012)	Deng et al.	N-list	FPS	BT
HUI-Miner (2012)	Liu et al.	Utility-list	HUPS	QT
MBIS-FWUI (2015)	Nguyen et al.	MBIS	FWUPS	QT
SHU-Growth (2016)	Ryang et al.	SHU-Tree	HUPS	Quantitative data streams
WUN-Miner (2016)	Bui et al.	WUN-set	FWUPS	QT
EFIM (2017)	Zida et al.	None	HUPS	QT
MAHUSP (2017)	Zihayat et al.	MAS-Tree	HUSPs	Sequences data streams
MPM (2018)	Yun et al.	DAT, TUL	HAUPS	Quantitative data streams
ILDHUP (2020)	Nam et al.	DUI list	HUPS	Quantitative data streams
MED (2020)	Baek et al.	MED-Tree	EPs	Product data streams
FWUP-SWUNL (2023)	Nguyen et al.	SWUN-list	FWUPS	QT
HUPM Stream (2023)	Han et al.	Ext-list	HUPS	Quantitative data streams
dFWUT (2023)	Nguyen et al.	Tidset	FWUPS	Dynamic QT
dFWUNL (2023)	Nguyen et al.	WUNList	FWUPS	Dynamic QT
FOTH (2024)	Yan et al.	IndexSet	HUPS	QT
DEPL (2024)	Kim et al.	dei-lists	EPs	Product data streams

3. Preliminaries

3.1. Problem definition

Let $I = \{i_j\}_{j=1}^n$ be the set of items. A set $X \subseteq I$ is called a pattern (or itemset), and a k -pattern if X contains exactly k items.

Definition 1. A transaction t_k in dQDB (referred to as a dynamic quantitative transaction) includes a set $t_k = \{x_{ij}\}_{j=1}^n$, where x_{ij} is the number of units bought of item $i_j \in I$ in transaction t_k and is associated with a collection of weights of the items $dw_i = \{w_j\}_{j=1}^n$. Each transaction is assigned a distinct identifier referred to as Tid .

In a dQDB, an item's weight can be adjusted at any time based on its significance. Consequently, the weights are organized into several batches $\{dw_j\}_{j=1}^p$, with each batch encompassing one or more sequential transactions.

Definition 2. A dynamic quantitative data stream $DS = \{t_j\}_{j=1}^m$ is a chronological list of dynamic quantitative transactions. DS does not limit the number of transactions, so transactions $t_{m+1}, t_{m+2}, \dots$ can be added after transaction t_m .

In the sliding window model, the values for window size and pane size are predetermined, where each pane holds a fixed number of transactions, and a window consists of multiple panes. Let $|P|$ represent the number of transactions in a pane and $|W|$ represent the number of transactions in a window. If w is the window size, $|W| = |P| * w$. A data stream (DS) is divided into multiple panes, represented as $DS = [P_1, P_2, \dots, P_l]$. To update the window, the oldest pane is removed and replaced with data from a new pane. This cycle continues until no additional data can be incorporated.

Example 1. Fig. 2 illustrates the sliding window model for a dynamic quantitative data stream DS_e , where both the window size and pane size are set at 2. DS_e has a total of six transactions $\{t_j\}_{j=1}^6$ and five items $\{A, B, C, D, E\}$ divided into two windows $\{W_1, W_2\}$. Each window includes two panes, and each pane contains two transactions. The weight of the items across the transactions is detailed in Table 2. The weights of items in transactions t_1, t_2, t_3 is $dw_1 = \{0.2, 0.4, 0.2, 0.4, 0.5\}$, and the weights of items in transactions t_4, t_5, t_6 is $dw_2 = \{0.5, 0.3, 0.4, 0.6, 0.6\}$.

Transaction	A	B	C	D	E	Weight
t_1	1	0	2	2	1	dw_1
t_2	0	2	1	2	2	
t_3	2	0	1	0	1	
t_4	1	0	3	2	0	dw_2
t_5	0	1	1	1	0	
t_6	2	0	2	3	1	

Fig. 2. An example of dynamic quantitative stream data (DS_e).

Definition 3. [9] The transaction weighted utility (twu) of a transaction t_k , according to a specific set of weights, dw_p , is calculated as follows:

$$twu(t_k) = \frac{\sum_{ij \in t_k} w_{pij} \times x_{kij}}{|t_k|}, \quad (1)$$

where $w_{pij} \in dw_p$ is the weight of ij ($ij \in t_k$), x_{kij} is the quantity of ij , and $|t_k|$ denotes the cardinality of t_k .

Example 2. All twu values in DS_e are calculated as shown in Table 3.

Definition 4. Let $sumtwu(W_i) = \sum_{t_k \in W_i} twu(t_k)$ be the sum of the twu values of the transactions in the window W_i . The weighted utility support (wus) of a pattern X in the i -th sliding window W_i is determined as follows:

$$wus(X, W_i) = \frac{\sum_{t_k \in t(X, W_i)} twu(t_k)}{sumtwu(W_i)}, \quad (2)$$

where $t(X, W_i)$ denotes the collection of transactions in the window W_i which contains the pattern X .

Example 3. The wus of AC of window W_1 in the DS_e is: $wus(AC) = [twu(t_1) + twu(t_3) + twu(t_4)] / sumtwu(W_1) = (0.48 + 0.37 + 0.97) / 2.52 \approx 0.72$.

The challenge of mining FWUPs from a dynamic quantitative data stream, employing the sliding window approach, involves identifying all FWUPs that attain a wus value that meets or exceeds a user-defined minimum weighted utility support threshold ($minwus$) within each data window.

3.2. Tidset data structure

The tidset data structure was first proposed by Zaki and Gouda [38] for mining FPs. Subsequently, many studies have applied this data structure to pattern mining problems and achieved significant efficiency. Nguyen et al. [9] also proposed an algorithm, named dFWUT, utilizing the tidset structure to mine FWUPs on dQDB. Fundamentally, the tidset of a pattern P , denoted as $t(P)$, is the set storing the Tid values of transactions contained P in the dQDB: $t(P) = \{t \in dQDB | P \subseteq t\}$. Based on tidset, we can quickly generate new patterns and determine whether a pattern is FWUP as follows.

Given two $(k-1)$ -patterns, Pu and Pv , with the same prefix P , where u and v are two items belonging to I . The tidset of two patterns Pu and Pv are $t(Pu)$ and $t(Pv)$, respectively. Puv is a new pattern formed from Pu and Pv with tidset $t(Puv)$ calculated as follows.

$$t(Puv) = t(Pu) \cap t(Pv) \quad (3)$$

The wus of the pattern Puv is determined based on $t(Puv)$ as.

$$wus(Puv) = \frac{\sum_{t_k \in t(Puv)} twu(t_k)}{sumtwu}. \quad (4)$$

Example 4. Consider a dQDB consisting of four transactions $\{t_1, t_2, t_3, t_4\}$ as shown in Fig. 2. We have item A and C have tidset $t(A) = \{1, 3, 4\}$ and $t(C) = \{1, 2, 3, 4\}$, respectively. The pattern AC formed from A and C has tidset $t(AC) = t(A) \cap t(C) = \{1, 3, 4\}$ and $wus(AC) = 0.72$.

3.3. WUNList data structure

The WUNList data structure is an extension of the N-list structure [29], proposed by Nguyen et al. [9], to efficiently mine FWUPs from dQDB. The WUNList is a list consisting of one or more WUN-codes, where each WUN-code is a tuple of three values: *pre*, *post*, and *weight*. The WUN-codes of a 1-FWUP are initially generated from the WUN-tree. The WUN-tree represents a hierarchical data structure characterized by a singular root node accompanied by an array of subordinate child nodes. Each node in the WUN-tree stores five fields: *name*, *weight*, *pre*, *post*, and *children*, which represent the name of the item, the cumulative transaction weighted utility associated with the node, the sequences of pre-order and post-order traversals associated with the node, along with the corresponding child nodes of this particular node. Fig. 3 illustrates the WUN-tree of the dQDB with four transactions $\{t_1, t_2, t_3, t_4\}$ (from Fig. 2) and $minwus = 0.5$. Item B is eliminated because $wus(B) < minwus$. Item A has WUN-code $\langle 3, 1, 1.45 \rangle$ and $\langle 6, 5, 0.37 \rangle$, thus the WUNList of item A is $wunl(A) = \{\langle 3, 1, 1.45 \rangle, \langle 6, 5, 0.37 \rangle\}$.

Table 2

The weight sets of items.

Weight	A	B	C	D	E
dw_1	0.2	0.4	0.2	0.4	0.5
dw_2	0.5	0.3	0.4	0.6	0.6

Table 3The twu values of all transactions in DS_e .

Transaction	Weight	Formula	twu
t_1	dw_1	$(0.2 \times 1 + 0.2 \times 2 + 0.4 \times 2 + 0.5 \times 1) / 4$	0.48
t_2	dw_1	$(0.4 \times 2 + 0.2 \times 1 + 0.4 \times 2 + 0.5 \times 2) / 4$	0.7
t_3	dw_1	$(0.2 \times 2 + 0.2 \times 1 + 0.5 \times 1) / 3$	0.37
t_4	dw_2	$(0.5 \times 1 + 0.4 \times 3 + 0.6 \times 2) / 3$	0.97
t_5	dw_2	$(0.3 \times 1 + 0.4 \times 1 + 0.6 \times 1) / 3$	0.43
t_6	dw_2	$(0.5 \times 2 + 0.4 \times 2 + 0.6 \times 3 + 0.6 \times 1) / 4$	1.05

The authors have presented the definitions and theorems to quickly generate new patterns by intersecting two WUNLists and calculating the wus value of a pattern from the information stored in the WUNList as follows.

Let I_1 be the set of items in I after sorting in descending order according to the weighted utility support of each item. Given two $(k-1)$ -patterns, Pu and Pv , with the WUNLists of $wunl(Pu)$ and $wunl(Pv)$, respectively. These two patterns share the same prefix P , where v precedes u in I_1 . The WUNList of the k -pattern Puv , $wunl(Puv)$, is created as follows: (1) For each WUN-code $C_i(\langle pre_i, post_i, weight_i \rangle) \in wunl(Pu)$ and $C_j(\langle pre_j, post_j, weight_j \rangle) \in wunl(Pv)$, if C_j is an ancestor of C_i , add $\langle pre_j, post_j, weight_i \rangle$ to $wunl(Puv)$. C_j is an ancestor of C_i if and only if $pre_j < pre_i$ and $post_j > post_i$. (2) Traverse $wunl(Puv)$ and merge WUN-codes with the same pre value into new WUN-code with a weight value equal to the sum of the weights of these WUN-codes.

The wus of the pattern P , $wus(P)$, with WUNList of $wunl(P) = \{\langle pre_j, post_j, weight_j \rangle\}_{j=1}^m$, is determined as follows.

$$wus(P) = \frac{\sum_{i=1}^m weight_i}{sumtwu}. \quad (5)$$

Example 5. From WUN-tree in Fig. 3, we have $wunl(C) = \{\langle 1, 6, 2.52 \rangle\}$ and $wunl(E) = \{\langle 4, 0, 0.48 \rangle, \langle 5, 2, 0.7 \rangle, \langle 7, 4, 0.37 \rangle\}$, with $wus(C) = 2.52/2.52 = 1$ and $wus(E) = (0.48 + 0.7 + 0.37)/2.52 \approx 0.62$, respectively. We can quickly calculate the WUNList of the pattern EC as $wunl(EC) = \{\langle 1, 6, 1.55 \rangle\}$ and $wus(EC) = 1.55/2.52 \approx 0.62$.

3.4. Overall framework

To address the challenges of mining FWUPs over dynamic quantitative data streams, we propose a continuous, pane-based sliding window framework. The high-level architecture of this framework is illustrated in Fig. 4. This framework provides an overview of the entire mining process, which consists of three primary stages: window management, FWUP mining, and result outputting.

The process begins with the window management stage, which activates upon the arrival of a new data pane from the stream. This stage first scans the data in the new pane and then constructs or updates the proposed data structures (the CTset and DSWUN-tree, described in Section 4 and Section 5). It ensures that all relevant data within the new window are completely and accurately maintained. Next, the FWUP mining stage is executed. This stage leverages the information stored in the updated data structures to recursively identify all patterns that satisfy the $minwus$ threshold. Once mining is complete, the process proceeds to the final result outputting stage, where the complete set of discovered FWUPs is provided to the user for the current window. The framework then iterates, looping back to the window management stage to await the next data pane. This update-then-mine architecture ensures that

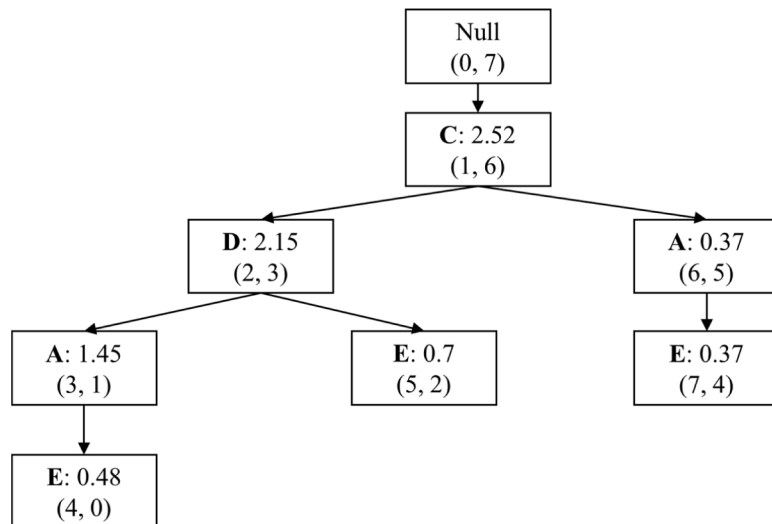


Fig. 3. The WUN-tree is created from W_1 of DS_e with $minwus = 0.5$.

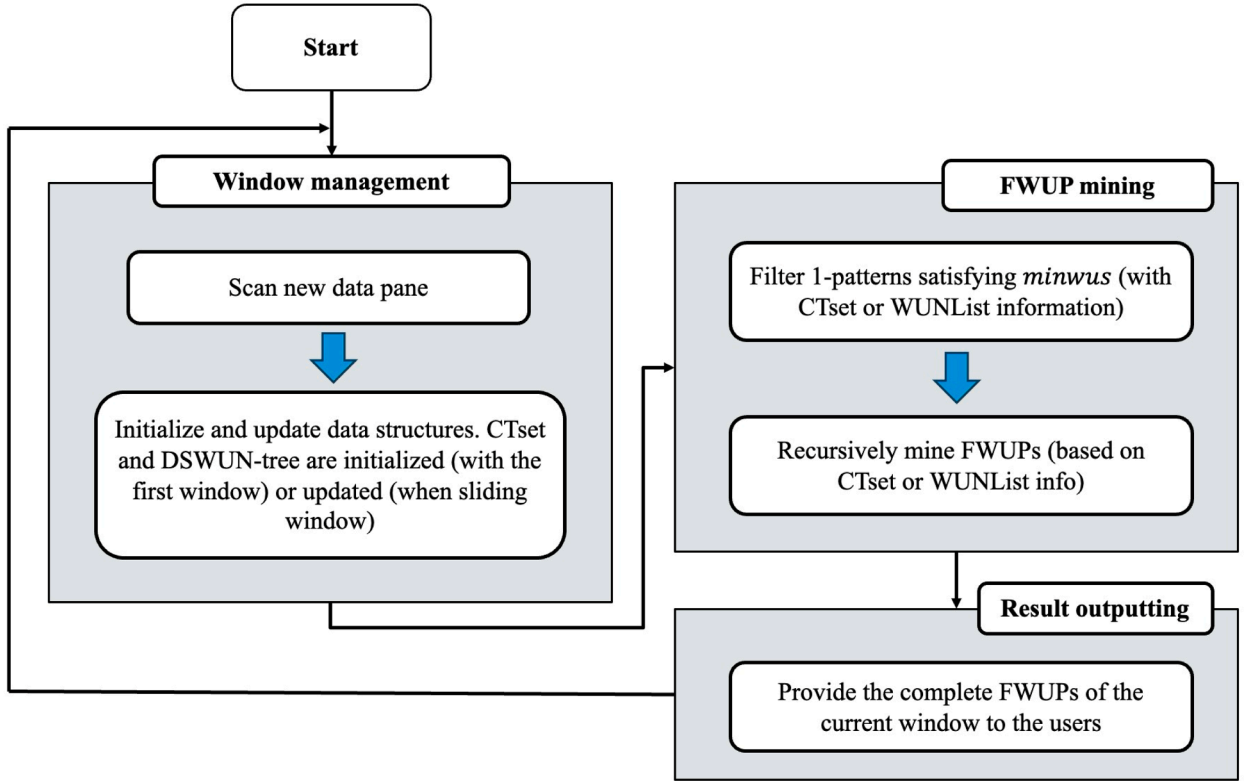


Fig. 4. Overall framework of the proposed method.

mining is always performed on the most up-to-date data, while the specialized data structures introduced in the following sections eliminate the need for expensive rescans of the complete window.

4. Mining FWUPs using Circular Tidset data structure over data streams

4.1. Circular Tidset data structure

The first proposed algorithm for mining FWUPs over dynamic quantitative data streams is named FWUDS-CT, which utilizes the Circular Tidset data structure. Although the tidset structure was initially designed for static databases with a fixed number of transactions, it could be adapted for data streams due to its single-scan nature. A notable limitation of the tidset structure, however, is that the Tid values can grow excessively large in data streams that feature an unlimited number of transactions, making the processing of large numbers both complex and resource-intensive. To address this, we developed a novel data structure called Circular Tidset, denoted as CTset, which is an improvement of the tidset structure, specifically designed to accommodate data streams more efficiently.

Definition 5. The local transaction identifier, denoted as $lcTid$, is the identification value of a transaction in the current window. Initially, $lcTid$ has a value of 0. If a new transaction is added, the $lcTid$ of that transaction is calculated as follows:

$$lcTid = \begin{cases} lcTid + 1, & \text{if } lcTid < |W| \\ 1, & \text{if } lcTid = |W| \end{cases} \quad (6)$$

where $|W|$ represents the total count of transactions in the window.

The local transaction identifier is used to uniquely identify transactions within the current sliding window, rather than across the entire data stream. At the beginning of the window, the $lcTid$ of the first transaction is initialized to 0 and incremented for each newly added transaction. Once the window reaches its maximum size, the sliding operation removes the oldest transaction and admits a new one, during which the $lcTid$ values are reused in a circular manner. This local indexing mechanism enables the rapid identification of transactions to be evicted or updated as the window slides, eliminating the need to maintain a global identifier. Consequently, it reduces search overhead, minimizes memory consumption, and enhances update efficiency in continuous data stream environments.

Example 6. Fig. 5 shows DS_e after adding the $lcTid$ values. Window W_1 has four transactions, t_1 , t_2 , t_3 , and t_4 , with $lcTid$ values of 1, 2, 3, and 4, respectively. Window W_2 has four transactions, t_3 , t_4 , t_5 , and t_6 , with $lcTid$ values of 3, 4, 1, and 2, respectively. We can easily see that the $lcTid$ value does not exceed $|W|$.

Definition 6. Circular Tidset, denoted as CTset, of a pattern X in window W_i is a list storing the $lcTids$ of transactions containing X in window W_i .

$$CTset(X, W_i) = \{lcTid(t) \in W_i | X \subseteq t\} \quad (7)$$

Example 7. According to Fig. 5, in window W_1 , pattern A appears in three transactions t_1 , t_3 , and t_4 , so $CTset(A, W_1) = \{1, 3, 4\}$. Similarly, in window W_2 , three transactions t_3 , t_4 , and t_6 contain pattern A , thus $CTset(A, W_2) = \{3, 4, 2\}$.

Definition 7. The wus value of a pattern X in window W_i is calculated based on $CTset(X, W_i)$ as follows:

$$wus(X, W_i) = \frac{\sum_{k \in CTset(X, W_i)} twu(t | lcTid(t, W_i) = k)}{sumtwu(W_i)}, \quad (8)$$

Where $lcTid(t, W_i)$ is the $lcTid$ value of transaction t in window W_i .

Example 8. From Example 7, we have $CTset(A, W_2) = \{3, 4, 2\}$, so $wus(A, W_2) = (twu(t_3) + twu(t_4) + twu(t_6)) / sumtwu(W_2) = (0.37 + 0.97 + 1.05) / 2.82 \approx 0.85$.

4.2. Maintaining CTsets over data stream

After the initialization of the first window, the sliding window process commences, involving changes to the data within the window: transactions from the oldest pane are removed, and new ones are added. This requires updating the CTset for each item to reflect the data of the current window. When adding a new pane into the window, the $lcTid$ for each new transaction is calculated according to Definition 5. Then, the $lcTid$ is added to the end of the CTset of the items appearing in that transaction. Consequently, the order of the newest transaction information in the CTset will be at the end of the list, while the beginning of the list will contain the $lcTid$ of the older transactions. The operation of removing the oldest pane from the window is more complex and is performed before the addition operation. We can determine the $lcTids$ of the oldest pane as follows.

Theorem 1. Suppose the $lcTid$ of the newest transaction in the current window has a value of n (the window is full), then the oldest pane in this window will contain transactions with $lcTid$ value in $[n+1, n+|P|]$ if $n < |W|$, and in $[1, |P|]$ if $n = |W|$.

Proof: Clearly, with $n = |W|$, the $lcTid$ value of the transactions in this window are sequentially $\{1, 2, \dots, n-1, n\}$, thus the oldest pane will include transactions with $lcTid$ as $[1, 2, \dots, |P|-1, |P|]$. Similarly, with $n < |W|$, the current window contains $lcTids$ sequentially $\{n+1, n+2, \dots, |W|, 1, 2, \dots, n\}$, so oldest pane includes $lcTids$ value $\{n+1, n+2, \dots, n+|P|\}$.

Based on Theorem 1, we traverse the CTset of all items appearing in the window from the beginning to the end of the list. For each item, if the $lcTids$ value belongs to the oldest pane, it will be removed from the CTset. Otherwise, we terminate the process for the current item and proceed to the next one, as the following transactions are certainly not from the oldest pane. Given the need to add and remove elements at both ends of the list, we use the linked list data structure to manage the CTset. The pseudocode for these two operations is shown in Fig. 6.

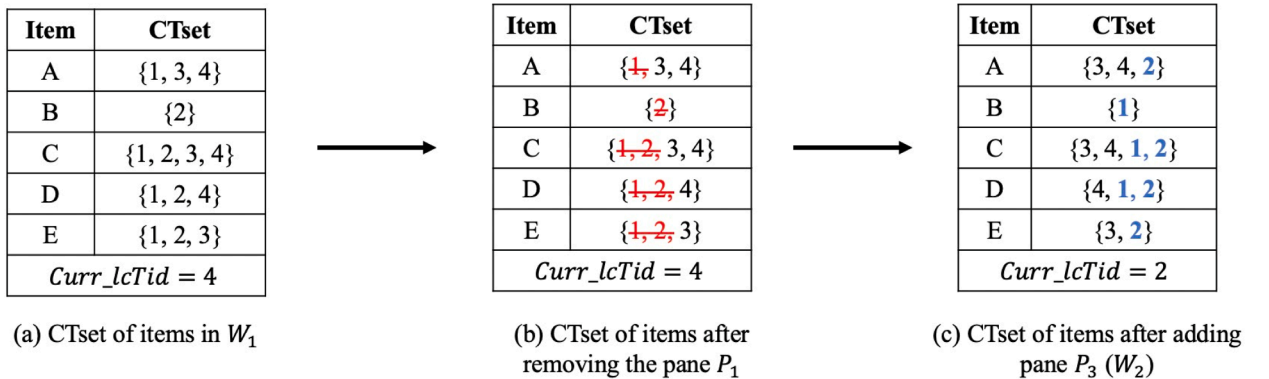
Example 9. The window W_1 in DS_e includes two panes with four transactions. The CTset of the items in W_1 is determined as shown in Fig. 7 (a), and the $lcTid$ value of the latest transaction in W_1 is $Curr_lcTid = 4$. The sliding window process from W_1 to W_2 updates the CTset of items as follows. First, we remove the oldest pane P_1 in W_1 . Since $Curr_lcTid = |W|$, the oldest pane includes $lcTid$ values of 1 and 2, which will be deleted from the CTset of items (Fig. 7(b)). Next, the new pane P_3 containing two transactions t_5 , t_6 is added with $lcTid$ values of 1 and 2, respectively. Transaction t_5 has items B, C, and D, so $lcTid(t_5) = 1$ is added to the end of the CTset for these items. Similarly, $lcTid(t_6) = 2$ is added to the CTset of items A, C, D, and E. At the end of the process, we obtain the CTset of the items in W_2 (Fig. 7(c)), and the new $Curr_lcTid$ value is 2.

		Transaction	Tid	lcTid	A	B	C	D	E	Weight
W_1 $(P_1 - P_2)$	P_1	t_1	1	1	1	0	2	2	1	dw_1
		t_2	2	2	0	2	1	2	2	
	P_2	t_3	3	3	2	0	1	0	1	
		t_4	4	4	1	0	3	2	0	
W_2 $(P_2 - P_3)$	P_3	t_5	5	1	0	1	1	1	0	dw_2
		t_6	6	2	2	0	2	3	1	

Fig. 5. DS_e With Tid and $lcTid$ values.

Algorithm 1. Adding and removing panes using CTset
Sub Procedure <i>InsertNewPane</i> ($P, Curr_lcTid, CTS, w$)
Input: A newest pane P , a current Local Transaction Identifier $Curr_lcTid$, a set of Circular Tidsets CTS , and a window size w
Output: the updated CTS for the new window
1. for each transaction t in P do 2. $Curr_lcTid \leftarrow Curr_lcTid + 1$ 3. if $Curr_lcTid > w * P $ then 4. $Curr_lcTid \leftarrow 1$ 5. for each item i in t do 6. Append $Curr_lcTid$ to $CTS[i]$
Sub Procedure <i>RemoveOldestPane</i> ($Curr_lcTid, CTS, w, p$)
Input: A current Local Transaction Identifier $Curr_lcTid$, a set of Circular Tidsets CTS , a window size w , and a pane size p
Output: the updated CTS for the new window
1. $rmFrom \leftarrow 1; rmTo \leftarrow p$ // Initialize the $lcTid$ range to be removed 2. if $Curr_lcTid < w * p$ then // Theorem 1 3. $rmFrom \leftarrow Curr_lcTid + 1$ 4. $rmTo \leftarrow Curr_lcTid + p$ 5. for each CTset CT_i in CTS do 6. for each element $lcTid$ in CT_i do 7. if $lcTid \geq rmFrom$ and $lcTid \leq rmTo$ then 8. Delete $lcTid$ from CT_i 9. else break

Fig. 6. Algorithm for adding and removing panes using CTset structure.

Fig. 7. The process of updating the CTset of items when sliding window from W_1 to W_2 in DS_e .

4.3. A linear method for intersecting CTsets

CTset stores transaction information of patterns in the current window. Similar to the tidset structure, the CTset of a new pattern can be determined by intersecting the CTsets of two patterns with the same prefix. With a native approach, the complexity of this intersection function is $O(mn)$, where m and n are the lengths of the two CTsets, respectively. The traditional tidset structure ensures increasing monotonicity, making it easy to intersect two tidsets with a complexity of $O(m + n)$. The CTset does not have this property. However, we have researched and developed a method to achieve linear complexity for the CTsets intersection function based on the following theorem.

Theorem 2. Let t_x and t_y be two different transactions in the current window with $lcTid$ values $lcTid_x$ and $lcTid_y$, respectively, and the $Curr_lcTid$ be the $lcTid$ value of the newest transaction in this window. Transaction t_y is newer than t_x when (1) $lcTid_x < lcTid_y \leq Curr_lcTid$, or (2) $Curr_lcTid < lcTid_x < lcTid_y$, or (3) $lcTid_x > Curr_lcTid$ and $lcTid_y \leq Curr_lcTid$.

Proof: With $Curr_lcTid = |W|$, the $lcTid$ values of transactions in the window in chronological order are $\{1, 2, \dots, Curr_lcTid - 1, Curr_lcTid\}$. It is evident that transaction t_y is more recent than transaction t_x if $lcTid_x < lcTid_y \leq Curr_lcTid$.

With $Curr_lcTid < |W|$, the $lcTid$ values of transactions in the window in chronological order are $\{Curr_lcTid + 1, Curr_lcTid + 2, \dots, |W|, 1, 2, \dots, Curr_lcTid\}$. In this case, there are two cycles within the window: $\{Curr_lcTid + 1, Curr_lcTid + 2, \dots, |W|\}$ belongs to the old cycle and $\{1, 2, \dots, Curr_lcTid\}$ to the new cycle. If transactions t_x and t_y both belong to the same cycle, then transaction t_y is newer than transaction t_x if $lcTid_x < lcTid_y \leq Curr_lcTid$ (same new cycle), or $Curr_lcTid < lcTid_x < lcTid_y$ (same old cycle). If the two transactions are from different cycles, transaction t_y is newer than transaction t_x if transaction t_y is in the new cycle, while transaction t_x is in the old cycle, or $lcTid_x > Curr_lcTid$ and $lcTid_y \leq Curr_lcTid$.

According to Theorem 2, the process of creating a CTset of a new pattern is effectively performed with linear complexity and is described in Fig. 8. The inputs for the CTsets intersection function are CTset CT_1 of pattern PX , CTset CT_2 of pattern PY , $Curr_lcTid$ as the $lcTid$ of the newest transaction in the window, and w and p as the values of window size and pane size, respectively. First, we initialize the CTset CT_3 of pattern PXY as empty, ptr_1 and ptr_2 store the first element of CT_1 and CT_2 , respectively (line 1–2). The algorithm sequentially compares the $lcTid$ values of ptr_1 and ptr_2 according to Theorem 2. To facilitate comparison, we employ a technique to increase the $lcTid$ by $|W| = w * p$ units if its value is less than or equal to $Curr_lcTid$. This indicates that this $lcTid$ belongs to the newly added transaction to distinguish it from older transactions with $lcTids$ greater than $Curr_lcTid$ (lines 4–8). Next, the two newly updated values $lcTid_1$ and $lcTid_2$ are compared according to the cases (lines 9–13):

- 1) If $lcTid_1 = lcTid_2$, we add $lcTid$ value of ptr_1 to CT_3 and then move to the next element of both CT_1 and CT_2 .
- 2) If $lcTid_1 < lcTid_2$, we move to the next element of CT_1 .
- 3) If $lcTid_1 > lcTid_2$, we move to the next element of CT_2 .

This process is continued until one of the two CTsets is fully traversed. The final result is the CTset of pattern PXY with a complexity of $O(m + n)$.

4.4. The FWUDS-CT algorithm

This section presents the FWUDS-CT algorithm for mining FWUPs on sliding window-based data streams (Fig. 9). When a new data

Algorithm 2. CTsets intersection
Input: Two Circular Tidset CT_1 and CT_2 , a current Local Transaction Identifier $Curr_lcTid$, a window size w , and a pane size p Output: The Circular Tidset CT_3
Procedure <i>CTsets_Intersection</i> ($CT_1, CT_2, Curr_lcTid, w, p$) 1. $CT_3 \leftarrow \emptyset$ 2. $ptr_1 \leftarrow CT_1.first; ptr_2 \leftarrow CT_2.first$ 3. while $ptr_1 \neq NULL$ and $ptr_2 \neq NULL$ do 4. $lcTid_1 \leftarrow ptr_1.lcTid; lcTid_2 \leftarrow ptr_2.lcTid$ 5. if $lcTid_1 \leq Curr_lcTid$ then //Adjust $lcTid_1$ and $lcTid_2$ for circular comparison 6. $lcTid_1 += w * p$ 7. if $lcTid_2 \leq Curr_lcTid$ then 8. $lcTid_2 += w * p$ 9. if $lcTid_1 = lcTid_2$ then 10. Append $ptr_1.lcTid$ to CT_3 11. $ptr_1 \leftarrow ptr_1.next; ptr_2 \leftarrow ptr_2.next$ 12. else if $lcTid_1 < lcTid_2$ then $ptr_1 \leftarrow ptr_1.next$ 13. else $ptr_2 \leftarrow ptr_2.next$ 14. return CT_3

Fig. 8. Intersection algorithm of two CTsets.

pane is added, the algorithm will return all FWUPs in the new window that satisfy the *minwus* threshold to the user. First, the algorithm updates the CTsets in the new window by removing the oldest pane using the **RemoveOldestPane** procedure (line 1) and adding the new pane using the **InsertNewPane** procedure (line 2). After updating data, we filter for 1-FWUPs, which are items with a *wus* value greater than or equal to *minwus*, stores these 1-FWUPs in C_1 , and calls the **MiningFWUPs_CTset** procedure to mine FWUPs (lines 3–6). The **MiningFWUPs_CTset** procedure is recursively called using the CTset data structure to mine FWUPs. The CTset of the candidate k -pattern X is formed from the $(k-1)$ -FWUPs with the same equivalent class stored in C (lines 4–5). If the *wus* of X is not less than *minwus*, X is an FWUP and is added to C_{next} to serve the following candidate generation process (lines 6–8). This process continues until no new patterns are generated. Finally, we obtain all the FWUPs from the most recent data of the window, and these tasks are repeated whenever a new pane is entered.

5. Mining FWUPs using DSWUN-tree data structure over data streams

5.1. The DSWUN-tree data structure

Nguyen et al. introduced the WUNList data structure [9] to discover FWUPs from dQDB, achieving superior experimental results in terms of both runtime and memory usage. The WUNList is generated from the WUN-tree structure, which is an extension of the N-list structure [29]. The WUN-tree compresses the database in tree nodes and eliminates infrequent items. However, the WUN-tree is limited to databases with a fixed number of transactions and requires up to two scans to be constructed. Building the WUN-tree is time-consuming, especially as the data size increases. Therefore, this structure is unsuitable to apply the sliding window model over data streams. To address this issue, we propose a new tree structure, named the DSWUN-tree, designed specifically to efficiently mine FWUPs within data streams.

Definition 8. The DSWUN-tree is a tree structure consisting of a root node and a set of child nodes. Each node in the DSWUN-tree includes six fields (*name*, *weight*, *pre*, *post*, *children*, and *parent*), where *name* represents the item corresponding to the node, *weight* is the total twu of transactions passing through the node, *pre* is the order of the node in preorder traversal, *post* is the order in post-order traversal, *children* is the list of child nodes, and *parent* is the parent of the current node.

Unlike the WUN-tree, the proposed DSWUN-tree structure additionally stores the parent node to facilitate tree updates when data

Algorithm 3. FWUDS-CT algorithm	
Input: A new pane P , a current Local Transaction Identifier $Curr_lcTid$, a set of Circular Tidsets of the current window CTS , a window size w , a pane size p , and a user-given threshold <i>minwus</i> ;	
Output: a set of FWUPs in the new sliding window	
1. RemoveOldestPane($Curr_lcTid, CTS, w, p$) 2. InsertNewPane($P, Curr_lcTid, CTS, w$) 3. Filter items with <i>wus</i> not less than <i>minwus</i> and store them in C_1 4. $FWUPs \leftarrow C_1$ 5. MineFWUPs_CTsets($C_1, Curr_lcTid, w, p$) 6. return $FWUPs$	
Procedure MineFWUPs_CTsets ($C, Curr_lcTid, w, p$) 1. for each c_i in C do 2. $C_{next} \leftarrow \emptyset$ // Initialize the next equivalence class 3. for each c_j in C ($j > i$) do 4. $X \leftarrow c_i \cup c_j$ // Generate a new k-pattern X from $(k-1)$-patterns c_i and c_j 5. $CT(X) \leftarrow CTsets_Intersection(CT(c_i), CT(c_j), Curr_lcTid, w, p)$ 6. if $wus(X) \geq minwus$ then 7. $FWUPs \leftarrow X$ 8. $C_{next} \leftarrow \{X, CT(X), wus(X)\}$ 9. if $C_{next} > 1$ then 10. MineFWUPs_CTsets($C_{next}, Curr_lcTid, w, p$)	

Fig. 9. FWUDS-CT algorithm.

changes. Moreover, the DSWUN-tree stores the entire database without removing infrequent items. A significant advantage of the DSWUN-tree is that it only requires a single database scan to update the tree, which can then be maintained and mined multiple times with various thresholds. However, the DSWUN-tree requires more memory for storage due to retaining more information. Additionally, we use an auxiliary linked list, named TAILLIST, to store the last node of each transaction added to the tree. We use this list to quickly update the tree when data changes. The algorithm for construction of the DSWUN-tree is described in Fig. 10.

Example 10. Fig. 11 illustrates DSWUN-tree created after adding the first four transactions of window W_1 in DS_e , denoted as DSWUN-tree (W_1). The transactions are reordered based on the twu values and added to the tree. Specifically, transaction t_1 is reordered as $\{C, D, A, E\}$, t_2 as $\{C, D, E, B\}$, t_3 as $\{C, A, E\}$, and t_4 as $\{C, D, A\}$. For each transaction added to the DSWUN-tree, the final node associated with this transaction is incorporated into the TAILLIST, resulting in TAILLIST having four tail nodes corresponding to the four transactions in W_1 .

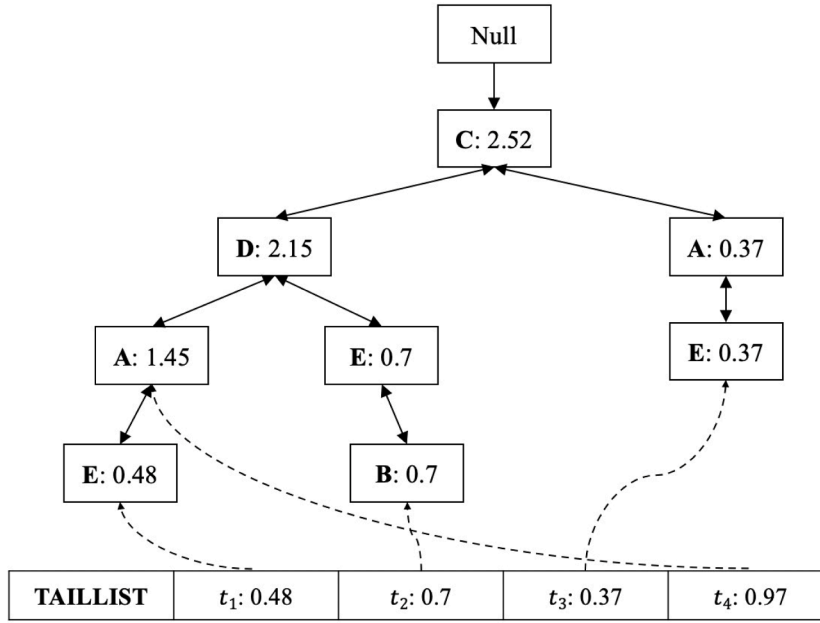
5.2. Updating the DSWUN-tree

In the sliding window phase, when a new pane is added, the oldest pane is removed from the DSWUN-tree because it is no longer in the current window. Utilizing the TAILLIST and the parent node information stored at each node, we can easily extract the transactions in the oldest pane and delete them from the DSWUN-tree. Fig. 12 describes the algorithm for updating the DSWUN-tree. First, each transaction in the newest pane is added to the DSWUN-tree using the **InsertTree** subprocedure (lines 1, 2). Then, we iterate $|P|$ times, corresponding to the $|P|$ transactions in the oldest pane. For each iteration, we consider the first node in the TAILLIST corresponding to transaction t_i . Next, we reduce the weight of the current node by the twu value of this transaction. If the weight of this node diminishes to zero, it will be removed from the tree. Then, we move up to its parent node. This process repeats until we reach the root node of the DSWUN-tree (lines 4–7). Finally, in line 8, we remove the first node from the TAILLIST and start the next iteration. At the end of the update process, we obtain the DSWUN-tree for the new window.

Example 11. Examine the mechanism of the sliding window transition from W_1 to W_2 in DS_e , where P_3 is the newest pane and P_1 is the oldest pane. The process of updating the DSWUN-tree is shown in Fig. 13. First, pane P_3 includes two transactions, t_5 and t_6 , which are added to the DSWUN-tree as shown in Fig. 13(a) and 13(b), respectively. Then, the transactions t_1 and t_2 of pane P_1 that are not in W_2 , thus being removed from the DSWUN-tree. For transaction t_1 , the tail node in the TAILLIST is $(E : 1.53)$, we decrease the weight of this node by $twu(t_1) = 0.48$, and the node is updated to $(E : 1.05)$. The algorithm moves up to its parent node $(A : 2.02)$ and reduces its weight by 0.48. We continue to decrease the weight by 0.48 on its ancestor nodes $(D : 3.63)$ and $(C : 4)$. We obtain the DSWUN-tree after removing transaction t_1 , as depicted in Fig. 13(c). The process of removing transaction t_2 is similarly performed and described in Fig. 13(d). We decrease the weight of the tail node $(B : 0.7)$ by $twu(t_2) = 0.7$. The remaining weight value is 0, hence this node is removed from the DSWUN-tree. Next, the parent node $(E : 0.7)$ is also removed from the DSWUN-tree because its remaining weight equals zero after decreasing it by 0.7. Finally, we traverse the two ancestor nodes $(D : 3.15)$ and $(C : 3.52)$ and update them to $(D : 2.45)$ and $(E : 2.82)$, respectively. The process completes, and we obtain the updated DSWUN-tree with the new data in window W_2 .

Algorithm 4. Construction of the DSWUN-tree	
Input: An initial window, IW	
Output: DSWUN-tree	
1. Create the root of DSWUN-tree, $R = 'Null'$ 2. for each transaction t in IW do 3. Sort t in descending order according to wus order of items in IW 4. InsertTree (t, R) 5. Return R	
Sub Procedure InsertTree (T, R)	
1. i_1 is the head of the sequence T 2. if exist node N in $R.child$ with $N.name = i_1.name$ then 3. $N.weight = N.weight + twu(T)$ 4. else 5. Append N as a child of R 6. $N.name \leftarrow i_1.name$; $N.weight \leftarrow twu(T)$; $N.parent \leftarrow R$ 7. if $ T = 1$ then Append N to TAILLIST // N is the leaf node 8. else InsertTree ($T/i_1, N$)	

Fig. 10. The dswun-tree construction algorithm.

Fig. 11. The DSWUN-tree is formed from W_1 of DS_e .

Algorithm 5. Update the DSWUN-tree
Input: DSWUN-tree R and a newest pane P
Output: the updated DSWUN-tree
Procedure Update_DSWUN-tree(R, P) 1. for $t \in P$ do 2. InsertTree (t, R) 3. for $i \leftarrow 1$ to $ P $ do // Process removing the oldest pane 4. Let L be the first element in $TAILLIST$ 5. for each node $N \neq \text{root}$ from L to root of R do 6. $N.\text{weight} -= \text{twu}(L.\text{tid})$ 7. if $N.\text{weight} = 0$ then delete N 8. Delete L in $TAILLIST$ 9. Return R

Fig. 12. The algorithm for updating DSWUN-tree.

5.3. The FWUDS-DWT algorithm

Using the techniques described above, the proposed algorithm, FWUDS-DWT, efficiently searches for FWUPs in sliding window-based data streams, as depicted in Fig. 14. Fundamentally, the operational mechanism of FWUDS-DWT is quite similar to the FWUDS-CT algorithm. First, in line 1, the algorithm calls the **Update_DSWUN-tree** procedure to update the data in the new window. The oldest pane in DSWUN-tree is removed, and a new pane is added to DSWUN-tree. Next, the FWUDS-DWT algorithm proceeds to find the 1-FWUPs, and storing them in I_1 (line 2). We traverse DSWUN-tree to derive the *pre* and *post* values associated with each individual node and creating the WUNList for these patterns in I_1 (line 3). Afterward, the **MineFWUPs_WUNList** procedure is called to mine all FWUPs that meet the *minwus* threshold in the current window (lines 4–5). **MineFWUPs_WUNList** uses a depth-first search method, combined with the WUNList data structure, to quickly identify the next FWUPs. The process ceases when no new patterns emerge, and the FWUPs are provided to the user. The algorithm then continues the sliding window process for the next pane.

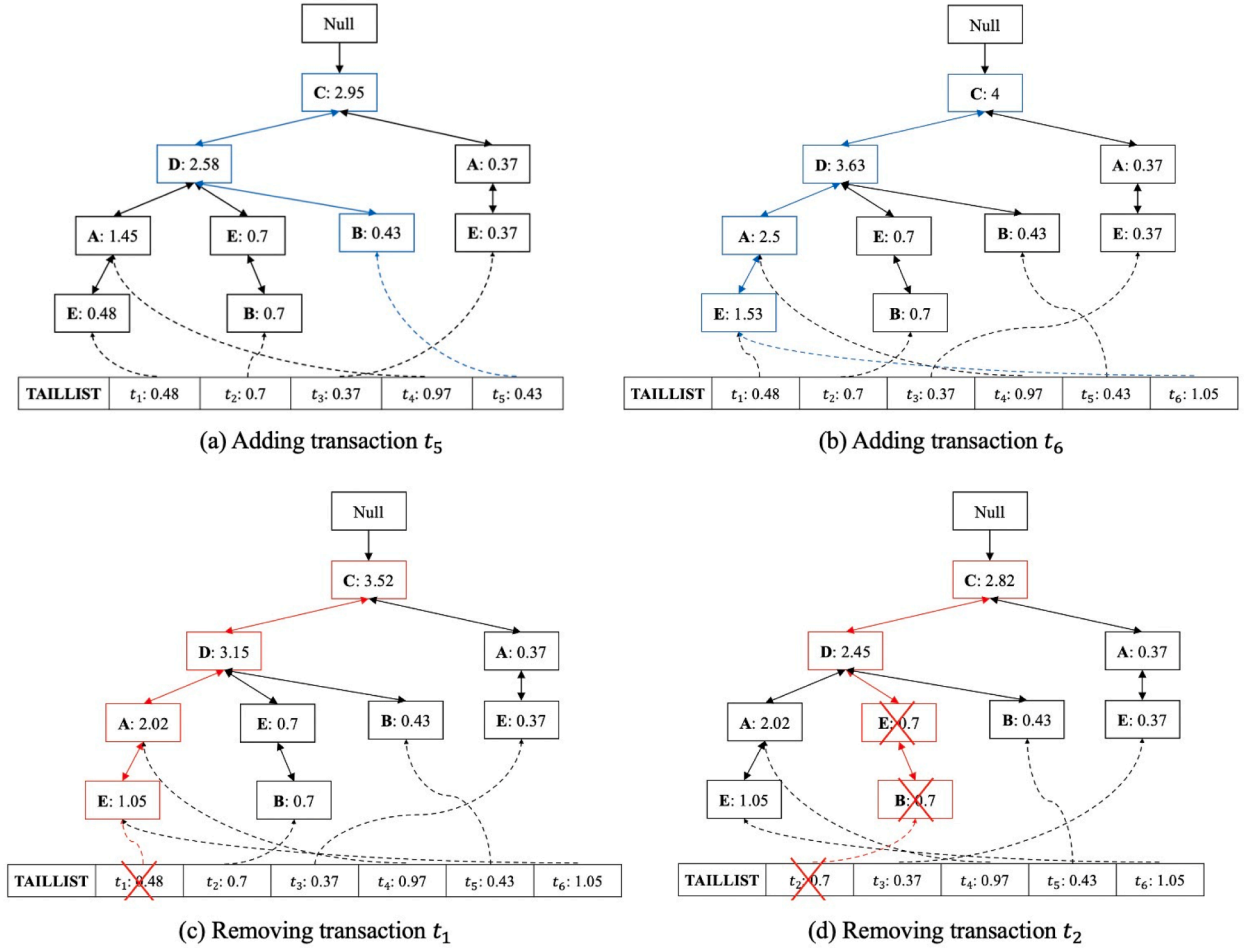


Fig. 13. The process of updating the DSWUN-tree from window W_1 to W_2 .

6. Evaluation

In this section, we present the empirical findings derived from the comparative analysis of two algorithms designed for mining FWUPs in dynamic quantitative data streams using the sliding window mechanism. The two proposed algorithms, FWUDS-CT and FWUDS-DWT, are compared with a modified version of the dFWUNL algorithm [9] (denoted as dFWUNL-stream). The dFWUNL algorithm is the current SOTA algorithm for mining FWUPs in dQDB. We have modified this algorithm to work on data streams while ensuring that performance and complexity are consistent with the original version. The experiments were executed to validate the practical effectiveness of the suggested theoretical enhancements for mining FWUPs on dynamic quantitative data streams. Specifically, we evaluated four key aspects: (1) the data update time during window sliding, which reflects the efficiency of each data structure; (2) the total execution time, indicating the overall computational cost; (3) the memory consumption, representing the space efficiency of the proposed models; and (4) scalability, assessing the performance of each algorithm under increasing data volumes.

6.1. Experimental settings

The dFWUNL-stream, FWUDS-CT, and FWUDS-DWT algorithms were implemented in C# 2022 on the .NET Framework 4.7.2 platform. All experiments were conducted on a Windows 11 machine with an Intel Core i5-9400F processor operating at 2.9 GHz and 32 GB of RAM. We also explicitly state that no parallelization techniques were used, and all experiments were run on a single thread. Table 4 summarizes the characteristics of the datasets used in these experiments. Retail, Accident, BMS-POS, and Kosarak were obtained from the FIMI repository (<https://fimi.uantwerpen.be/data/>). Susy was downloaded from the open-source library SPMF (<https://www.philippe-fournier-viger.com/spmf/>). To emulate dynamic quantitative data streams, each dataset was partitioned into multiple batches of equal size, with the final batch containing any remaining transactions. Item weights were randomly assigned for each batch, with values drawn from the interval [1,10].

Due to the varying sizes and characteristics of the experimental datasets, the window sizes, pane sizes, and *minwus* thresholds were

Algorithm 6. FWUDS-DWT Algorithm

Input: A new pane P , DSWUN-tree of the current window R , and a user-given threshold $minwus$;

Output: A set of **FWUPs** in the new sliding window

1. $R \leftarrow \text{Update_DSWUN-tree}(P, R)$
2. Store items that satisfy the $minwus$ in I_1
3. Generate the WUNList for these items in I_1
4. $FWUPs \leftarrow I_1$
5. **MineFWUPs_WUNList**(I_1)
6. **return** $FWUPs$

Procedure MineFWUPs_WUNList(C)

1. **for** $i \leftarrow |C| - 1$ **downto** 1 **do**
2. $C_{next} \leftarrow \emptyset$;
3. **for** $j \leftarrow i - 1$ **downto** 0 **do**
4. $X \leftarrow C[i] \cup C[j]$
5. $wunl(X) \leftarrow \text{WUNList_Intersection}(wunl(C[i]), wunl(C[j]))$
6. **if** $wunl(X) \neq \text{NULL}$ **then**
7. $FWUPs \leftarrow X$
8. $C_{next} \leftarrow \{X, wunl(X), wus(X)\}$
9. **if** $|C_{next}| > 1$ **then**
10. **MineFWUPs_WUNList**(C_{next})

Procedure WUNList_intersection($wunl_1, wunl_2$)

1. $wunl_3 \leftarrow \emptyset, AW \leftarrow 0$ // Initialize result WUNList and accumulated wus value
2. $i \leftarrow 0, j \leftarrow 0$ // Initialize pointers for $wunl_1$ and $wunl_2$
3. $WR_1 = wus(PX); WR_2 = wus(PY)$ // Initialize Wus-Remaining (WR) for pruning
4. **while** $i < |wunl_1| \wedge j < |wunl_2|$ **do**
5. **if** $wunl_2[j].pre < wunl_1[i].pre$ **then** // Check for ancestor-descendant
6. **if** $wunl_2[j].post > wunl_1[i].post$ **then** // $wunl_2[j]$ is an ancestor
7. **if** $|wunl_3| > 0 \wedge wunl_3[|wunl_3| - 1].pre = wunl_2[j].pre$ **then**
8. $wunl_3[|wunl_3| - 1].weight += wunl_1[i].weight$ // Accumulate the weight into the last ancestor entry
9. **else** // New ancestor node
10. $wunl_3 \leftarrow (wunl_2[j].pre, wunl_2[j].post, wunl_1[i].weight)$
11. $AW += wunl_1[i].weight; WR_1 -= wunl_1[i].weight; i++$ // Update AW, WR_1 , and move i
12. **else** $WR_2 -= wunl_2[j].weight; j++$ // Post-order check failed, update WR_2 and move j
13. **else** $WR_1 -= wunl_1[i].weight; i++$ // Pre-order check failed, update WR_1 and move i
14. **if** $AW + \min(WR_1, WR_2) < (minwus \times sumtwu)$ **then return NULL** // Abort if max possible wus is below threshold
15. **return** $wunl_3$

Fig. 14. FWUDS-DWT algorithm.

Table 4

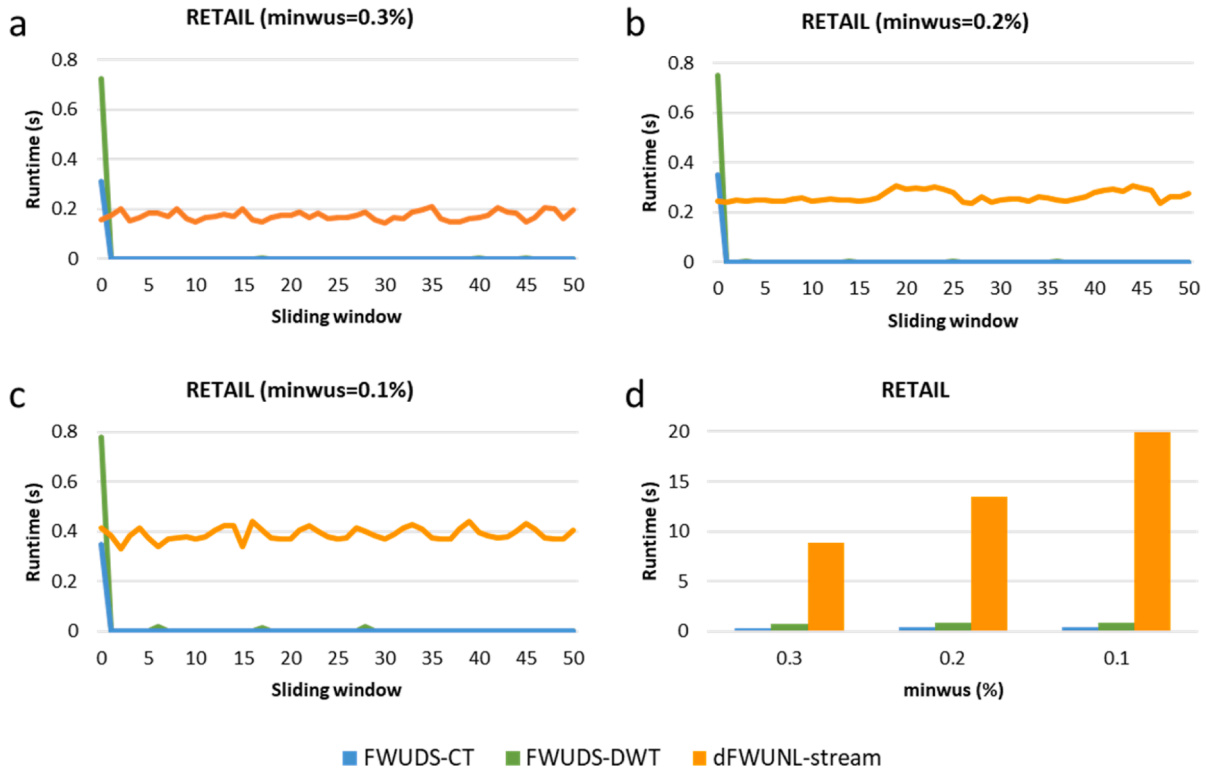
The features of the experimental datasets.

Dataset	# Items	# Trans	Avg. length	Type	# Trans of each batch	Description
Retail	16,470	88,162	10.3	Sparse	10,000	E-commerce customer transactions
Accidents	468	340,183	33.8	Dense	5,000	Anonymized traffic accident data
BMS-POS	1,657	515,597	6.5	Sparse	50,000	Large-retailer Point-of-Sale data
Kosarak	41,270	990,002	8.1	Sparse	100,000	News portal clickstream data
Susy	190	5,000,000	19	Dense	200,000	High-energy physics data

tailored for each dataset. This guarantees a distinct assessment of performance among the various algorithms. The experimental process on each dataset is conducted as follows. First, the algorithms sequentially scan the panes in the first window, build a data structure to store them and mine FWUPs within that window. Following this, the sliding window mechanism is activated: the oldest pane is removed, and a new pane is added. Subsequently, the FWUPs in the updated window are mined. All tests were conducted by sliding the window 50 times. Since three algorithms effectively identify all FWUPs, their performance is evaluated based on runtime, memory usage, and scalability. The runtime was measured as the total elapsed time from the initiation of the algorithm to its completion. Meanwhile, the memory consumption was determined based on the peak amount of memory utilized by the algorithm during its execution.

6.2. Runtime for data updates

First, we executed a series of experiments to assess the ability of the algorithms to construct structures and update data during the sliding window process. This is a critical task that significantly affects the efficiency of the algorithms, especially given the speed and continuity of the data stream. Figs. 15–19 show the results on experimental datasets. The algorithms were executed on three different *minwus* thresholds for each dataset, with a fixed pane size of 1. The total time for the data update process was then aggregated. Fig. 15 illustrates the findings derived from the Retail dataset with a window size of 88,000P (panes) and *minwus* thresholds of 0.3%, 0.2%, and 0.1%, respectively. The chart in Fig. 15(d) shows that the FWUDS-CT and FWUDS-DWT algorithms took significantly less time for data construction and updates compared to the dFWUNL-stream algorithm, with the FWUDS-CT algorithm having the shortest runtime. Furthermore, as the *minwus* threshold decreases, the runtime of the dFWUNL-stream algorithm increases, while the two proposed algorithms maintain nearly stable runtimes. This is because the dFWUNL-stream algorithm needs to continuously scan the database to create the WUN-tree for each new window. In contrast, the two proposed methods require time only to construct the CTsets and

**Fig. 15.** Runtime for data updates on the Retail dataset with varying *minwus* thresholds.

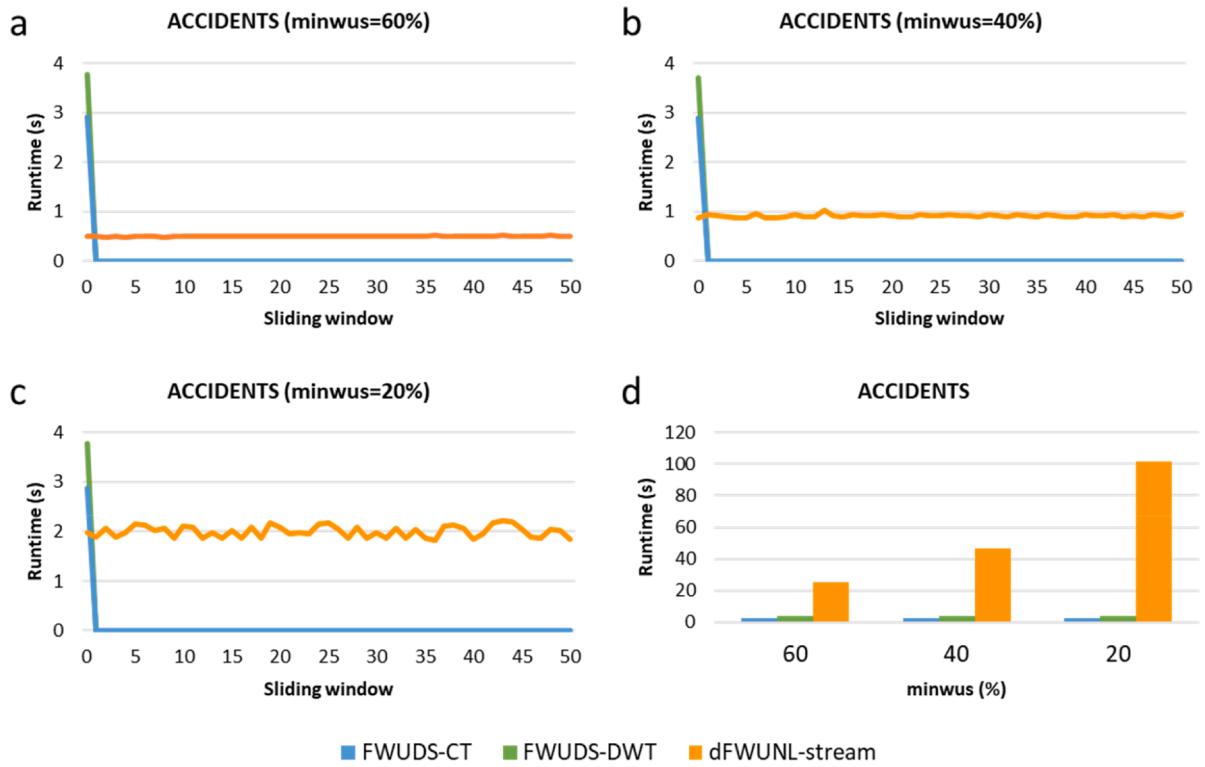


Fig. 16. Runtime for data updates on the Accidents dataset with varying *minwus* thresholds.

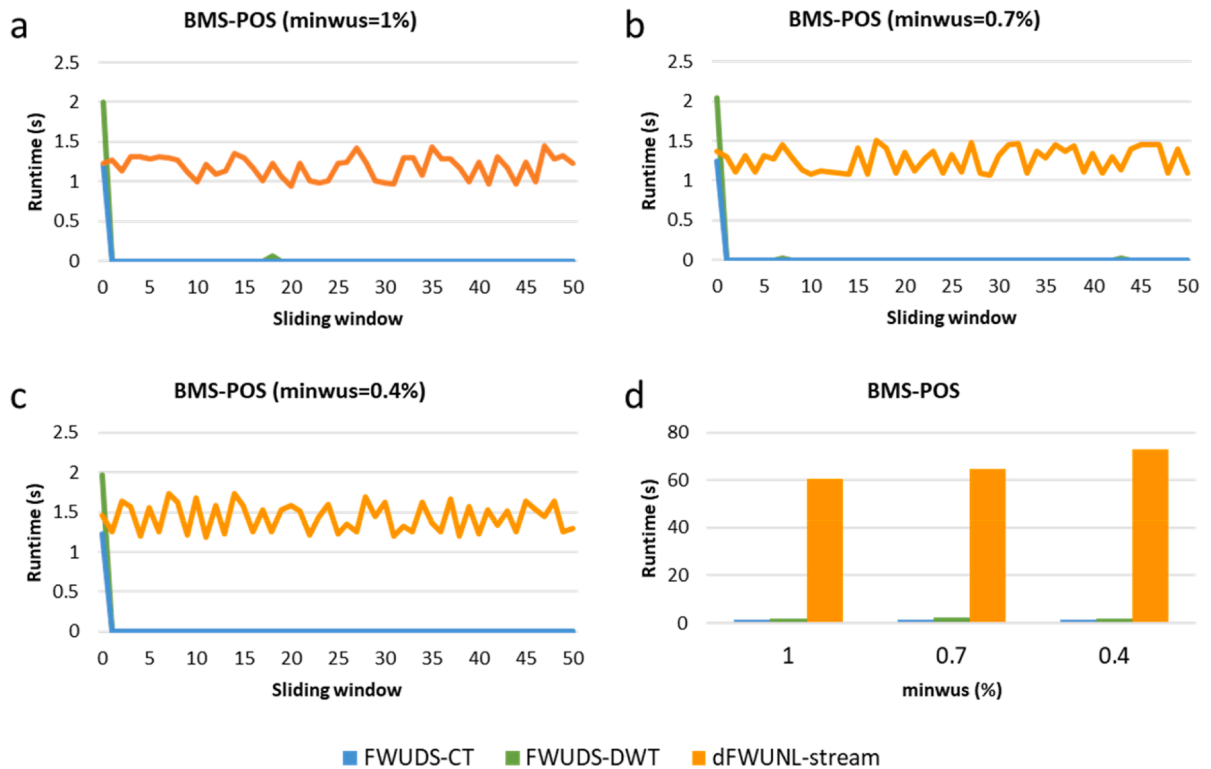


Fig. 17. Runtime for data updates on the BMS-POS dataset with varying *minwus* thresholds.

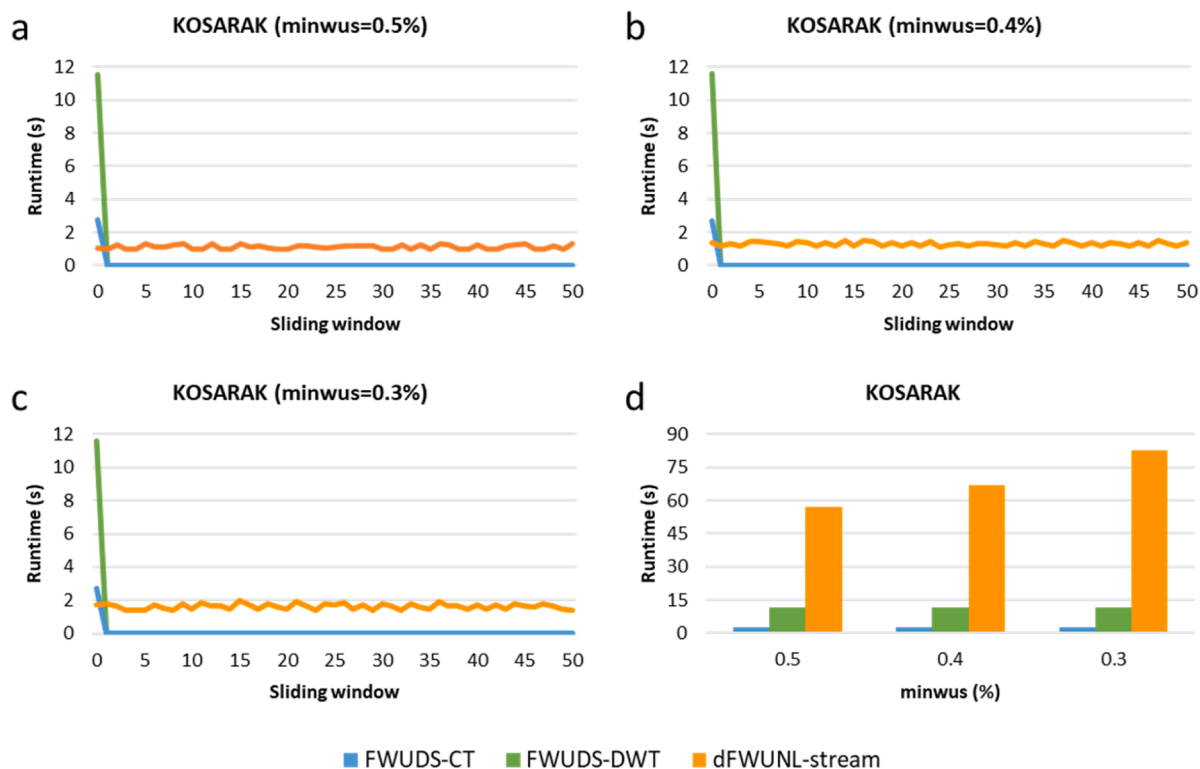


Fig. 18. Runtime for data updates on the Kosarak dataset with varying *minwus* thresholds.

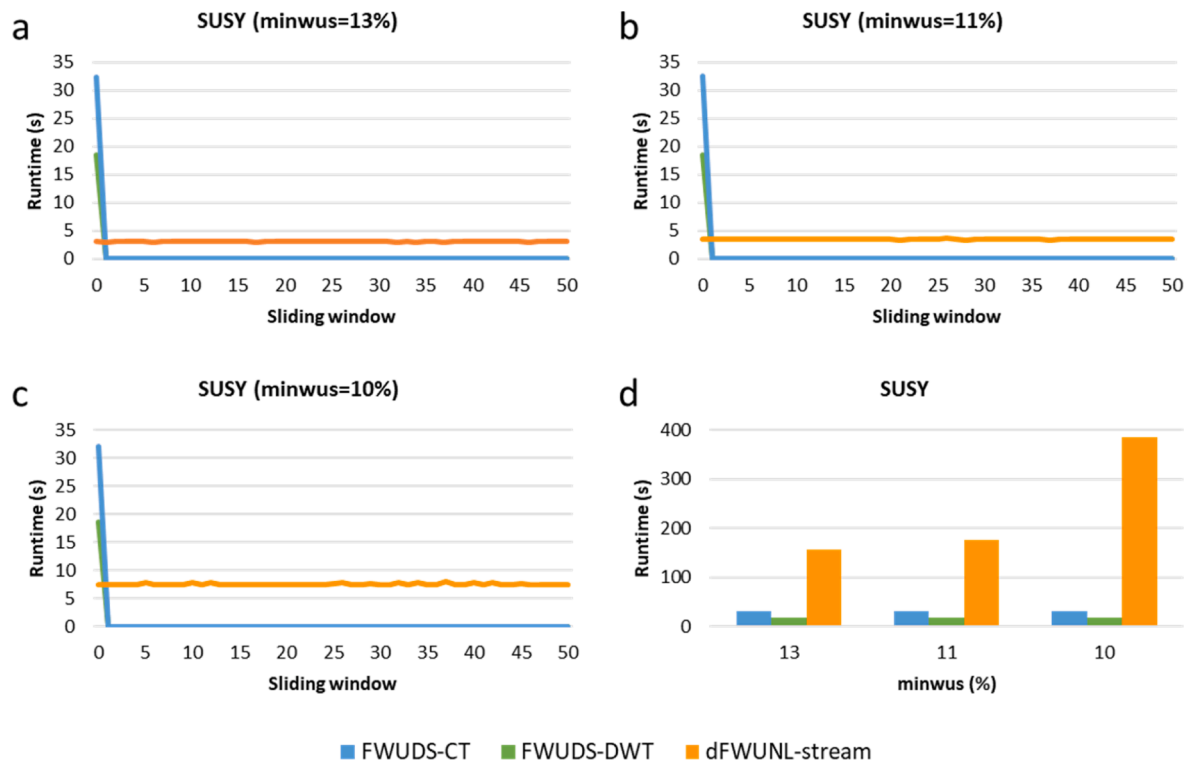


Fig. 19. Runtime for data updates on the Susy dataset with varying *minwus* thresholds.

DSWUN-tree structures for the first window, with almost no additional time needed for the sliding window process thereafter. The data from the old window is reused, with the update process simply removing the oldest pane and adding the newest one. As a result, after 50 times sliding windows, the total runtime of the FWUDS-CT and FWUDS-DWT algorithms is superior to that of the dFWUNL-stream algorithm. Similar results were observed for the Accidents, BMS-POS, Kosarak, and Susy datasets, with window sizes of 340,000P, 515,000P, 980,000P, and 4,000,000P, respectively. Notably, as the window size increases, the difference in the total time required for data construction and updates between the algorithms becomes more pronounced.

6.3. Runtime for FWUP mining

Fig. 20 shows the total mining time of FWUPs for the FWUDS-CT, FWUDS-DWT, and dFWUNL-stream algorithms on datasets with varying *minwus* thresholds. The mining time is measured from the start of the algorithm until it terminates. The window size and pane size are fixed, as described in Section 6.2. Due to the significant difference in runtime between the algorithms, we use a logarithmic scale ($\log_2$) for the runtime to facilitate comparison and presentation on the chart. The results show a trend where, as the *minwus* threshold decreases, the mining time of the algorithms increases. This is because, as the threshold lowers, the number of FWUPs increases, causing the algorithms to take more time to find all the satisfying patterns. Although the FWUDS-CT algorithm is efficient in data construction and updates, it takes more time to mine FWUPs. Moreover, with very small thresholds or very large datasets, such as Susy, the FWUDS-CT algorithm fails due to memory overflow. In contrast, the FWUDS-DWT algorithm demonstrates superiority, achieving the shortest runtime among the three algorithms tested. This is due to the efficiency of data updates using the DSWUN-tree structure, combined with the optimization of the mining process through the WUNList structure. For example, on the Retail dataset with a *minwus* threshold of 0.2%, the FWUDS-CT algorithm took 223 s, the dFWUNL-stream algorithm took 25 s, while the FWUDS-DWT algorithm took only 13.5 s. On the BMS-POS dataset with a threshold of 0.4%, the total mining times of the FWUDS-CT, FWUDS-DWT, and dFWUNL-stream algorithms after 50 times sliding windows were 845 s, 105.8 s, and 174.9 s, respectively.

6.4. Memory usage

To ensure a comprehensive evaluation, we conducted a systematic analysis of the peak memory consumption of the proposed algorithms compared with the baseline algorithm. This analysis is particularly crucial, as memory efficiency is often a primary constraint in data stream processing. The peak memory consumption (measured in megabytes, MB) was recorded for all algorithms across five datasets under varying *minwus* thresholds, as illustrated in Fig. 21. The window size and pane size parameters were configured as described in Subsection 6.2.

The experimental results highlight distinct differences and inherent design trade-offs among the algorithms. Overall, as the *minwus* threshold decreases, a greater number of FWUPs satisfy the condition, leading to increased memory usage across all methods. The FWUDS-DWT algorithm consistently exhibits the highest memory usage across all datasets. Specifically, its memory consumption remains nearly constant for certain datasets, approximately 1500 MB on Susy and 835 MB on Kosarak, regardless of the *minwus* value. This stability arises because the FWUDS-DWT algorithm maintains the entire DSWUN-tree structure in main memory, representing all transactions within the current sliding window. Furthermore, the DSWUN-tree structure is inherently complex and information-intensive, requiring substantial storage space. Although this design incurs considerable memory overhead, it enables FWUDS-DWT to achieve high runtime efficiency and support rapid data updates.

Conversely, the FWUDS-CT algorithm demonstrates superior memory efficiency on sparse datasets but performs less effectively on dense data. On the Retail and BMS-POS datasets, it is the most memory-efficient method, consuming only about 38.1 MB on Retail at *minwus* = 0.03, compared with 222 MB for FWUDS-DWT and 134.9 MB for dFWUNL-stream. This efficiency arises from its lightweight CTset data structure, which minimizes memory allocation in sparse data environments. However, when applied to dense datasets, the size of the item transaction identifier lists grows rapidly, resulting in significant memory expansion. Consequently, FWUDS-CT exhibits excessive runtime on Accidents at low thresholds and fails to execute on the large Susy dataset due to out-of-memory errors.

Meanwhile, the baseline algorithm, dFWUNL-stream, also demonstrates notable memory efficiency across datasets. This advantage stems from its strategy of constructing the WUN-tree only for items satisfying the *minwus* threshold and discarding it after generating the final WUN-list in each independent window. In contrast, both proposed algorithms retain all window data to support rapid updates. For instance, on the Retail dataset, FWUDS-CT consumes 33.4 MB, FWUDS-DWT consumes 110.4 MB, whereas dFWUNL-stream requires only 25.7 MB.

To furnish a thorough evaluation of efficacy regarding both execution duration and memory consumption, Table 5 summarizes the comparative results, reporting the percentage changes in runtime and memory usage of the proposed algorithms relative to the dFWUNL-stream baseline for each dataset. Positive values (+%) indicate performance improvements, reflected as reductions in runtime or memory usage. Negative values (–%) indicate performance degradation, corresponding to increases in runtime or memory consumption relative to the baseline.

6.5. Scalability

This section analyzes the scalability of the algorithms utilizing two extensive datasets, namely Kosarak and Susy. The evaluation is conducted by examining the efficiency of the algorithms as the window size and pane size gradually increase. Fig. 22 shows the scalability results for the data construction and update process on the Kosarak dataset, with the *minwus* threshold fixed at 0.2%. The window sizes were set to 700P, 750P, and 800P, with pane sizes of 600, 800, and 1000, respectively. The results indicate that as the

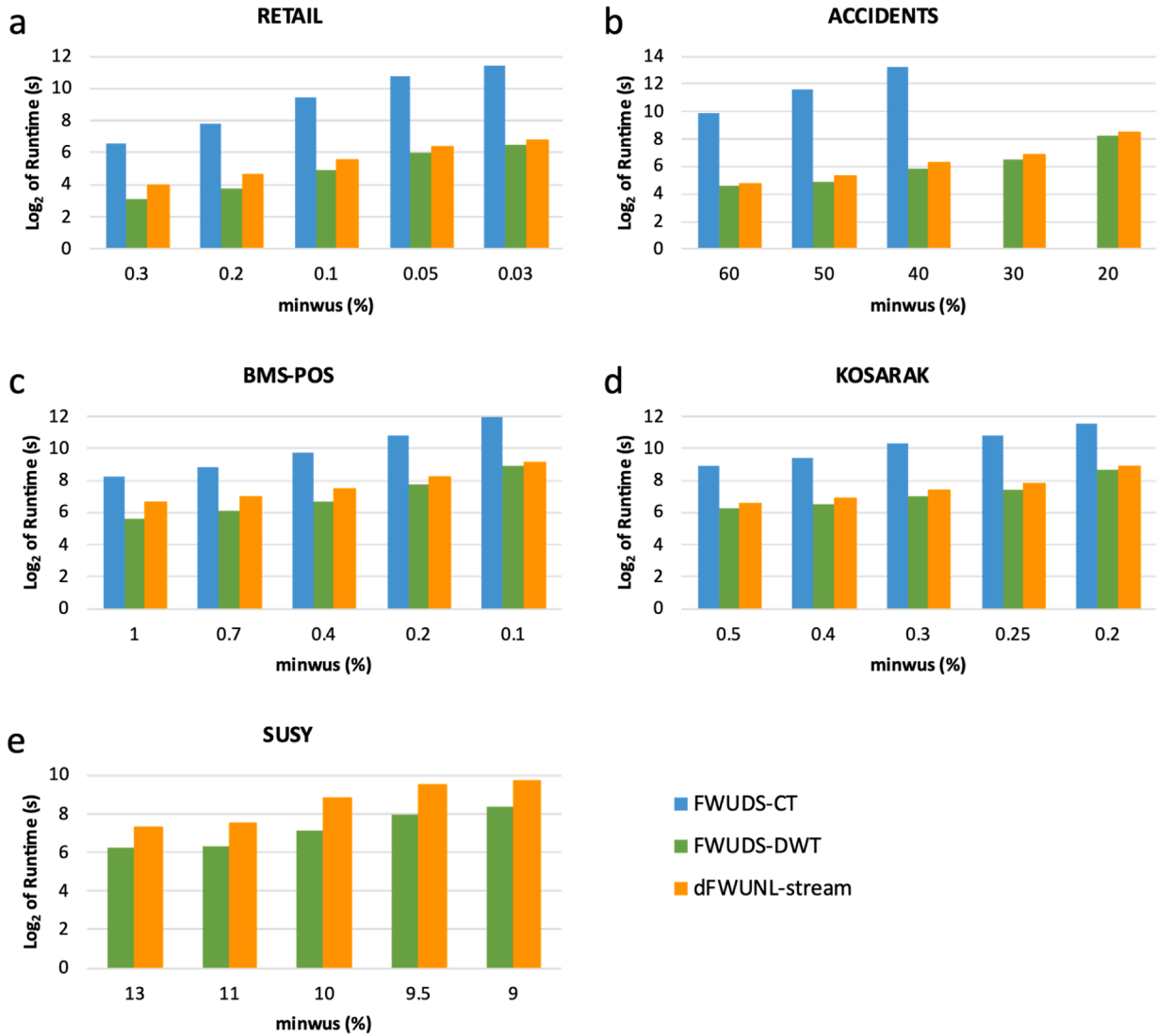


Fig. 20. Total runtime for FWUP mining on the experimental datasets with varying *minwus* thresholds.

window size and pane size increase, the data construction and update times of the algorithms also increase. According to the charts, the update times for the CTsets-based and DSWUN-tree-based methods increase slowly, while the dFWUNL-stream algorithm's update time increases rapidly with larger sizes. Fig. 23 shows the data update times on the Susy dataset with a *minwus* threshold of 10% after 50 times sliding windows. The window sizes and pane sizes were set to 800P – 3000, 900P – 3500, and 1000P – 4000, respectively. Similar to the Kosarak dataset, the results on the Susy dataset also demonstrate the stability of the FWUDS-CT and FWUDS-DWT algorithms during the data update process, even as the window size and pane size increase. In Fig. 23(d), with a window size of 800P and a pane size of 3000, the FWUDS-CT algorithm takes a total update time of 14 s, FWUDS-DWT takes 12.1 s, while the dFWUNL-stream algorithm takes 220.6 s. As the window size increases to 900P and the pane size to 3500 transactions, the FWUDS-CT algorithm takes 23.6 s, FWUDS-DWT takes 15.3 s, and dFWUNL-stream takes 294.8 s. Fig. 24 shows the scalability results in terms of total mining time for the algorithms on both datasets. As the window and pane sizes increase, the number of transactions in each window also increases, leading the algorithms to take more time to identify all FWUPs. The FWUDS-DWT algorithm exhibits a more stable increase in time compared to the other two algorithms. Therefore, we can confirm that the FWUDS-DWT algorithm has superior scalability when the size of the data stream changes.

Fig. 25 presents the memory scalability results of the algorithms under identical experimental conditions. On the sparse Kosarak dataset, the memory consumption of all three algorithms, FWUDS-CT, dFWUNL-stream, and FWUDS-DWT, increases nearly linearly with window size. This behavior aligns with theoretical expectations, as storing more transactions in memory requires additional space for their corresponding data structures. Among the three, FWUDS-DWT exhibits the highest absolute memory usage, which can be attributed to its design of maintaining the entire DSWUN-tree structure representing all transactions within the current window. The

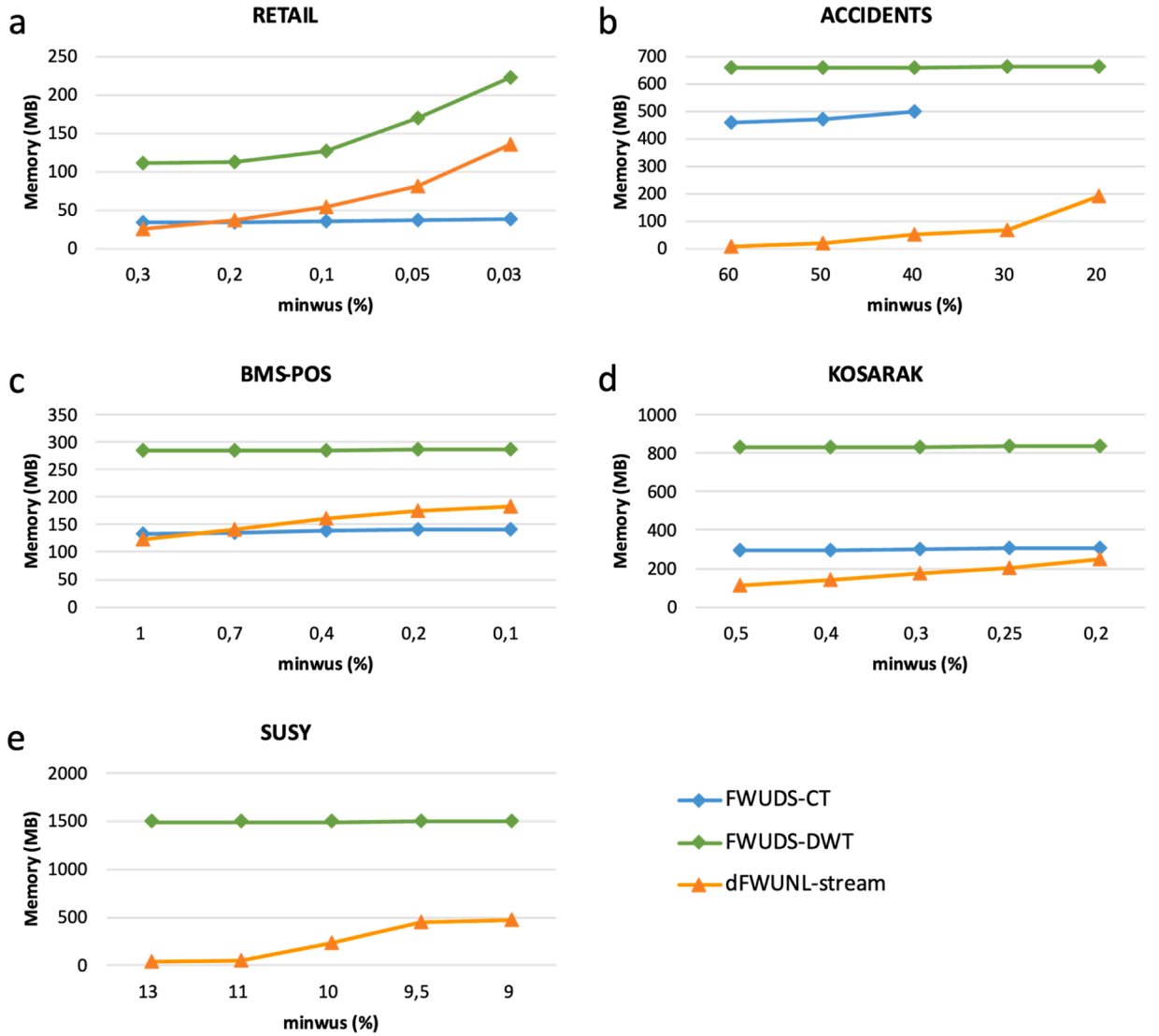


Fig. 21. Memory usage for FWUP mining on the experimental datasets with varying *minwus* thresholds.

Table 5

Summary of runtime and memory performance changes compared to the baseline.

Dataset	<i>minwus</i> (%)	Runtime reduction (%)		Memory reduction (%)	
		FWUDS-CT	FWUDS-DWT	FWUDS-CT	FWUDS-DWT
Retail	0.1%	−1,469.8%	33.5%	33%	−138.8%
Accidents	40%	−12,205.8%	29.3%	−870.8%	−1,186.2%
BMS-POS	0.2%	−489.9%	27.1%	19.7%	−64%
Kosarak	0.2%	−544.7%	15%	−25.2%	−240%
Susy	11%	N/A	55.9%	N/A	−214.1%

results on the dense Susy dataset provide further insight into the effect of data density on memory scalability. As previously noted, FWUDS-CT fails to execute under this condition due to out-of-memory errors. FWUDS-DWT's memory usage scales from 1024.3 MB to 1500.9 MB as the window size increases. Notably, the baseline dFWUNL-stream algorithm demonstrates the best memory scalability in this scenario, with memory usage increasing only slightly—from 184 MB to 235.6 MB. This outcome can be explained by the relatively high *minwus* threshold applied to the Susy dataset, which limits the number of frequent items and keeps the WUN-tree structure compact even as the input data volume grows. Overall, the increase in FWUDS-DWT's memory consumption is acceptable given its superior runtime efficiency.

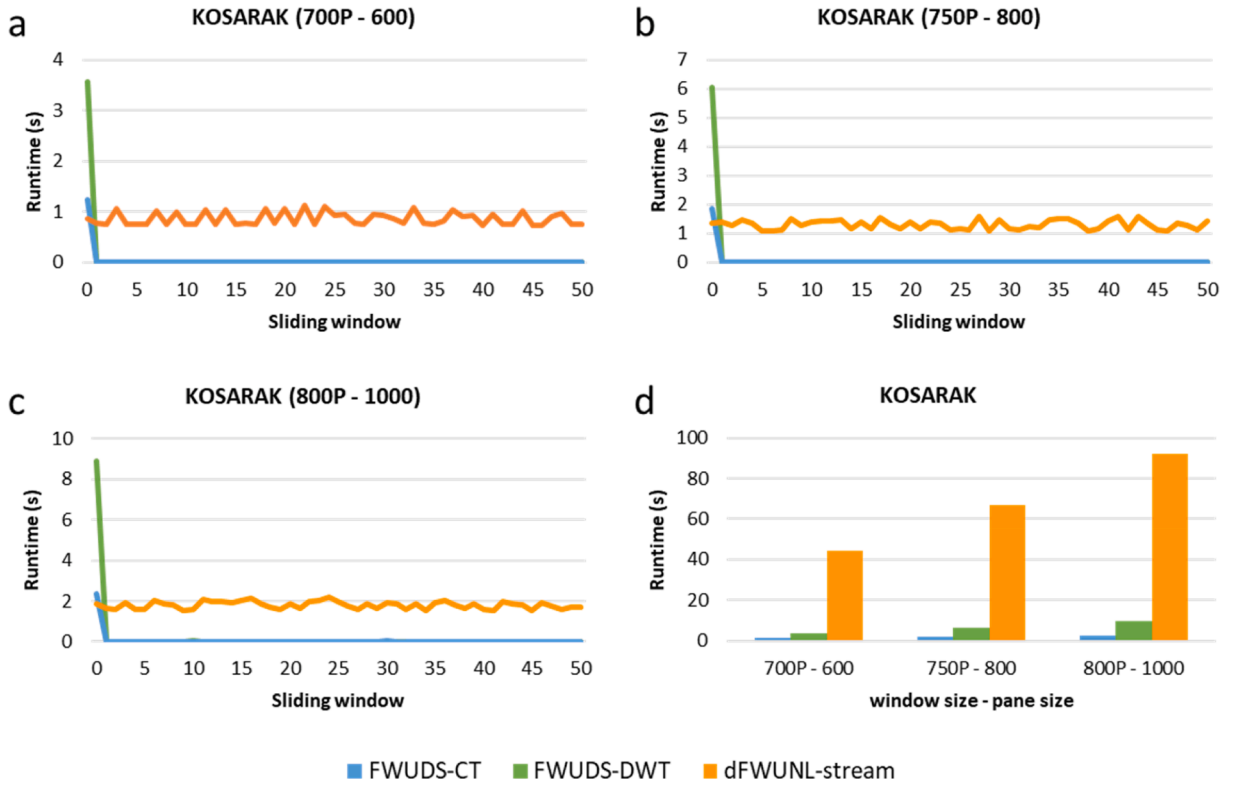


Fig. 22. Runtime for data updates on the Kosarak dataset with varying window sizes and pane sizes.

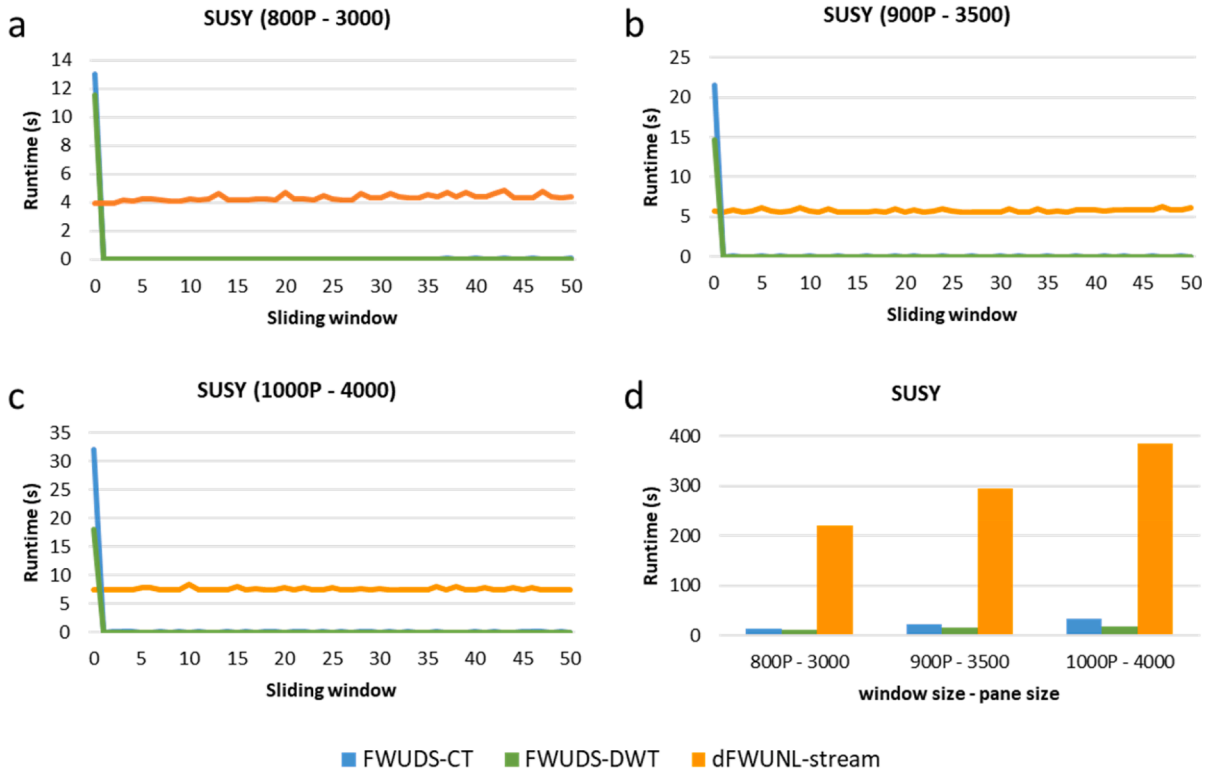


Fig. 23. Runtime for data updates on the Susy dataset with varying window sizes and pane sizes.

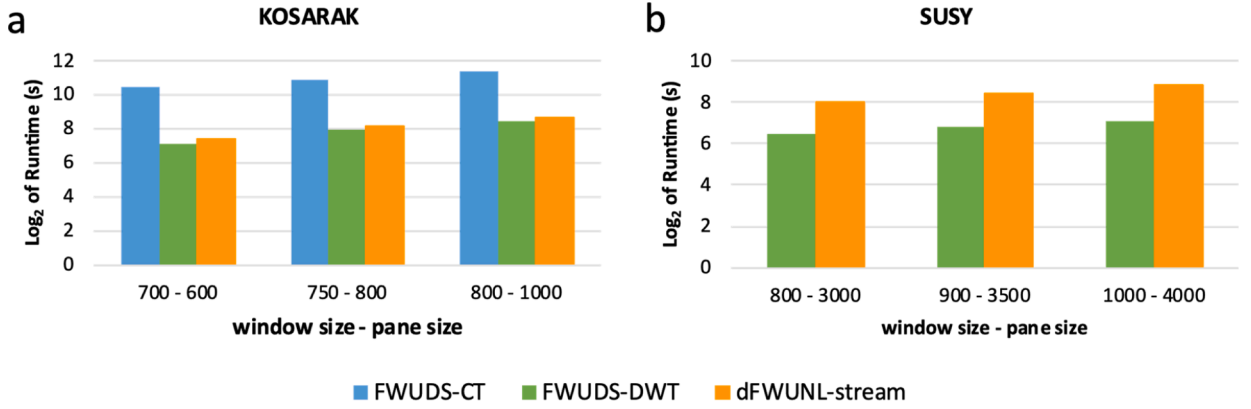


Fig. 24. Runtime scalability of FWUP mining with varying window and pane sizes on the Kosarak and Susy datasets.

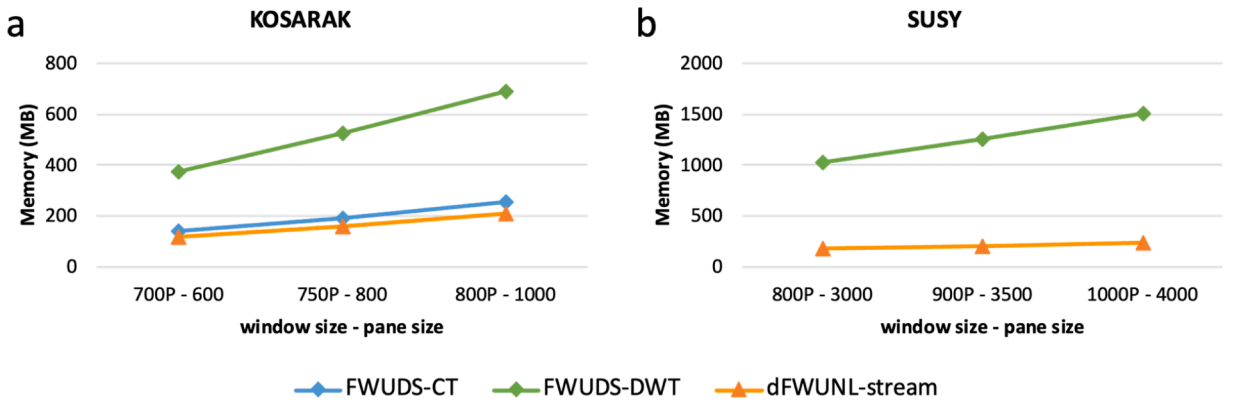


Fig. 25. Memory usage scalability of FWUP mining with varying window and pane sizes on the Kosarak and Susy datasets.

7. Complexity analysis and discussion

In this section, we elucidate the intricate complexity analysis pertaining to the methodologies that have been proposed. Based on the analytical results, we also discuss the inherent limitations of the algorithms under various real-world data stream scenarios. The time complexity of the FWUDS-CT algorithm consists of two primary components: window maintenance and mining. When a new pane of size p arrives, the window update process is executed using the **InsertNewPane** and **RemoveOldestPane** procedures (the latter is invoked when the window is full). The **InsertNewPane** procedure iterates through p transactions, each containing up to l_{max} items, where l_{max} denotes the maximum transaction length in the current window. Each item requires an *lcTid* insertion operation with a constant cost of $O(1)$, leading to a time complexity of $O(p \times l_{max})$. For the **RemoveOldestPane** procedure, the algorithm iterates through m CTsets, where m represents the number of unique items in the window. The total number of *lcTid* checking and deletion operations across all m lists is bounded by $O(p \times l_{max})$, resulting in an overall cost of $O(m + p \times l_{max})$. Therefore, the total time complexity for window maintenance per new pane is $O(m + p \times l_{max})$. Regarding the mining cost, the process begins with a recursive call to **MineFWUPs_CTsets**. The core operation involves generating candidate patterns and computing their CTsets via the **CTsets-Intersection** procedure. This intersection, performed linearly on two CTsets with a maximum size of ct_{max} , has a time complexity of $O(ct_{max})$. Let I_1 denote the set of items satisfying the *minwus* threshold. In the worst case, the algorithm explores the entire search space, where the total number of candidate patterns is $2^{|I_1|}$. Consequently, the overall time complexity of the mining phase is $O(ct_{max} \times 2^{|I_1|})$.

For the FWUDS-DWT algorithm, the initialization of the DSWUN-tree involves processing $n = w \times p$ transactions in the first window, where w denotes the number of panes and p the pane size. For each transaction, all items are sorted and inserted into the tree. Thus, the one-time initialization cost of the tree is $O(n \times l_{max} \times \log(l_{max}))$. For each incoming pane, the total time cost consists of tree update and mining phases. The update phase, performed by the **Update_DSWUN-tree** procedure, includes two operations: inserting p new transactions and evicting p obsolete ones. Inserting p new transactions requires p calls to **InsertTree**, each costing $O(l_{max} \times \log(l_{max}))$, resulting in a total cost of $O(p \times l_{max} \times \log(l_{max}))$. The eviction phase also iterates p times, and each iteration traverses from a leaf node in the **TAILLIST** up to the root, incurring $O(l_{max})$ time per operation. Therefore, the total tree update time per pane is $O(p \times l_{max} \times \log(l_{max}))$. The mining phase begins with generating the WUNList for the items in I_1 , which takes $O(|I_1| \times wunl_{max})$, where $wunl_{max}$ represents the maximum WUNList length. Subsequently, the recursive procedure **MineFWUPs_WUNList** is invoked.

The FWUDS-DWT algorithm explores a set-enumeration tree to mine FWUPs. In the worst case, the search space is $O(2^{|I_1|})$, corresponding to $2^{|I_1|}$ nodes. At each node, an intersection of two WUNLists is performed to generate a new pattern's WUNList, with a time complexity of $O(wunl_{max})$. Consequently, the overall time complexity for the mining process per window is $O(wunl_{max} \times 2^{|I_1|})$.

Based on the theoretical analysis, both proposed algorithms exhibit highly efficient data update performance during window sliding. The key reason lies in their pane-based incremental update mechanism, which eliminates the need to reprocess all n transactions in the sliding window, a requirement of the baseline dFWUNL-stream algorithm. Regarding mining time complexity, both algorithms theoretically possess exponential cost in the worst-case scenario. This is a common characteristic of pattern mining algorithms. However, in practice, the FWUDS-DWT algorithm demonstrates superior performance. This is attributed to FWUDS-DWT's use of the WUNList structure, which features data compression and progressively shortens after each intersection, thereby accelerating the pattern generation process compared to the CTset structure. In terms of space complexity, both algorithms must retain the complete data of the current window, requiring $O(w \times p \times l_{max})$ memory. However, the DSWUN-tree structure is structurally more complex than CTset and also includes a *TAILLIST* structure requiring $O(n)$ memory. Consequently, the DSWUN-tree demands more memory in practice, particularly on sparse datasets. Furthermore, both algorithms employ a depth-first search strategy for FWUP mining. The corresponding space complexity for this phase is $O(ct_{max} \times |I_1|^2)$ for FWUDS-CT and $O(wunl_{max} \times |I_1|^2)$ for FWUDS-DWT. As a result, on large and dense datasets, where items appear in numerous transactions, the FWUDS-CT algorithm tends to consume significantly more memory during mining.

Thus, the FWUDS-DWT algorithm sacrifices additional memory in exchange for its compressed DSWUN-tree structure and concise WUNLists, enabling faster mining performance. In contrast, the FWUDS-CT algorithm emphasizes an extremely lightweight and simple CTset structure but incurs a higher computational cost due to longer CTsets. This trade-off directly determines their respective application scenarios, limitations, and scalability. Specifically, FWUDS-DWT is the optimal choice for systems with abundant memory resources and high-frequency mining requirements, as its data compression substantially reduces the cost of WUNList intersections, resulting in superior runtime efficiency. Conversely, the FWUDS-CT algorithm is more suitable for applications where the input data is predominantly sparse, benefiting from its compact structure and minimal memory footprint. In terms of limitations and scalability, the bottlenecks of the two algorithms differ fundamentally. For FWUDS-DWT, the core limitation is memory. The requirement to maintain the DSWUN-tree structure for the entire current window causes memory to grow, limiting scalability for very large windows on sparse datasets. Meanwhile, the limitation of FWUDS-CT lies in processing time, as its mining performance degrades substantially when ct_{max} increases or when the *minwus* threshold is set too low on dense datasets.

Furthermore, the proposed methods exhibit several additional limitations related to scalability and real-world edge cases. First, under extremely high-velocity data streams, the update phase remains highly efficient and capable of real-time operation. However, the mining phase is inherently time-consuming. When the data ingestion rate becomes excessively high, the system may face a situation where updates occur in real time, but mining cannot keep pace. Consequently, the generated patterns may lag behind the most recent data, reducing the system's responsiveness to rapid changes. This challenge with high-dimensionality is not unique to pattern mining but is a well-known problem across the broader field of data mining. Second, for high-dimensional data, characterized by an extremely large number of unique items, the FWUDS-CT algorithm is directly affected, as it must maintain CTsets for all items. As the number of dimensions increases, this results in considerable resource overhead for both storage and updates, thereby limiting its scalability in such environments. Third, the current framework does not incorporate an explicit mechanism for handling noisy data. In real-world data streams (e.g., those affected by sensor errors or transmission noise), this limitation may lead to spurious patterns or distort weighted utility calculations. While robust noise-filtering or noise-tolerant mining techniques were beyond the scope of this study, we acknowledge this as an important limitation and identify it as a promising avenue for future research. Finally, another edge case arises from rapidly fluctuating weighted utility. The DSWUN-tree achieves high compression by ordering items according to their *wus*-order. However, frequent or abrupt changes in these weights can disrupt this ordering. As a result, the updated DSWUN-tree may exhibit lower compression efficiency, leading to an increase in both tree size and WUNList length, thereby degrading overall performance.

8. Conclusion

This paper introduces the problem of mining FWUPs from dynamic quantitative data streams using a sliding window model, where item weights can be adjusted. We propose two algorithms, FWUDS-CT and FWUDS-DWT, specifically designed to manage the dynamic changes in item weights and the continual influx of new data efficiently. Both algorithms employ novel data structures that facilitate the removal of outdated data panes, the integration of new panes, and the re-utilization of the existing panes within the sliding window, enabling FWUP mining through a single scan of the data. The FWUDS-CT algorithm employs a Circular Tidset data structure, an enhanced version of the tidset structure, that enables the rotation of transaction identifier values to avoid large numbers processing when data streams become very large. The FWUDS-DWT algorithm compresses data into a tree-based structure called DSWUN-tree, which enables quick data updates and generates a WUNList for patterns, thus optimizing the mining process. We have performed a series of experiments to assess these two algorithms across various datasets. The results indicate that the FWUDS-DWT algorithm significantly outperforms other algorithms in terms of runtime and scalability, though this efficiency comes at the cost of a bit higher memory consumption. Moving forward, we plan to extend our research to include mining frequent weighted utility closed patterns and top- k FWUPs in data streams. Additionally, we aim to explore the use of parallel programming architectures to further improve the performance of the algorithms and scalability for processing large-scale data. Another critical research direction involves adapting the proposed algorithms to real-time environments with fluctuating data inflow rates. While the current pane-based sliding window model

operates effectively under stable streams, we will investigate adaptive mechanisms to handle irregular or bursty data inflows. This will involve exploring solutions such as dynamically adjusting pane sizes or integrating load-shedding strategies to maintain real-time performance under varying stream conditions.

CRedit authorship contribution statement

Nguyen Le: Writing – original draft. **Ham Nguyen:** Conceptualization, Writing – review & editing. **Minh Nguyen:** Data curation. **Huong Bui:** Writing – review & editing, Visualization. **Bay Vo:** Writing – review & editing. **Unil Yun:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

References

- [1] R. Agrawal, T. Imieliński, A. Swami, Mining association rules between sets of items in large databases, in: Presented at the ACM sigmod record, ACM, 1993, pp. 207–216.
- [2] P. Fournier-Viger, J. Chun-Wei Lin, T. Truong-Chi, R. Nkambou, A survey of high utility itemset mining, in High-Utility Pattern Mining, vol. 51, P. Fournier-Viger, J.C.-W. Lin, R. Nkambou, B. Vo, V.S. Tseng, Eds., in Studies in Big Data, vol. 51., Cham: Springer International Publishing, 2019, pp. 1–45. doi: 10.1007/978-3-030-04921-8_1.
- [3] S. Kim, et al., Efficient approach for mining high-utility patterns on incremental databases with dynamic profits, Knowl.-Based Syst. 282 (2023) 111060, <https://doi.org/10.1016/j.knosys.2023.111060>.
- [4] C. Lee, et al., Efficient approach of sliding window-based high average-utility pattern mining with list structures, Knowl.-Based Syst. 256 (2022) 109702, <https://doi.org/10.1016/j.knosys.2022.109702>.
- [5] H. Kim, D.-W. Choi, Recency-based sequential pattern mining in multiple event sequences, Data Min. Knowl. Disc. 35 (1) (2021) 127–157, <https://doi.org/10.1007/s10618-020-00715-7>.
- [6] N.D. Ham, V.D. Bay, N.T.H. Minh, T.-P. Hong, MBIS: an efficient method for mining frequent weighted utility itemsets from quantitative databases, J. Comput. Sci. Cybern. 31 (1) (2015) 17–30.
- [7] H. Bui, B. Vo, H. Nguyen, WUN-miner: a new method for mining frequent weighted utility itemsets, Presented at the 2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2016, pp. 001365–001370.
- [8] H. Nguyen, N. Le, H. Bui, T. Le, A new approach for efficiently mining frequent weighted utility patterns, Appl. Intell. 53 (1) (2023) 121–140, <https://doi.org/10.1007/s10489-022-03580-7>.
- [9] H. Nguyen, N. Le, H. Bui, T. Le, Mining frequent weighted utility patterns with dynamic weighted items from quantitative databases, Appl. Intell. 53 (16) (2023) 19629–19646, <https://doi.org/10.1007/s10489-023-04554-z>.
- [10] M. Han, M. Li, Z. Chen, H. Wu, X. Zhang, High utility pattern mining algorithm over data streams using ext-list, Appl. Intell. 53 (22) (2023) 27072–27095, <https://doi.org/10.1007/s10489-023-04925-6>.
- [11] M. Li, M. Han, Z. Chen, H. Wu, X. Zhang, FCHM-stream: fast closed high utility itemsets mining over data streams, Knowl. Inf. Syst. 65 (6) (2023) 2509–2539, <https://doi.org/10.1007/s10115-023-01831-8>.
- [12] U. Yun, G. Lee, E. Yoon, Efficient high utility pattern mining for establishing manufacturing plans with sliding window control, IEEE Trans. Ind. Electron. 64 (9) (2017) 7239–7249, <https://doi.org/10.1109/TIE.2017.2682782>.
- [13] U. Yun, G. Lee, E. Yoon, Advanced approach of sliding window based erasable pattern mining with list structure of industrial fields, Inf. Sci. 494 (2019) 37–59, <https://doi.org/10.1016/j.ins.2019.04.050>.
- [14] Y. Liu, W. Liao, A. Choudhary, A two-phase algorithm for fast discovery of high utility itemsets, in: Presented at the Pacific-Asia Conference on Knowledge Discovery and Data Mining, Springer, 2005, pp. 689–695.
- [15] V.S. Tseng, C.-W. Wu, B.-E. Shie, P.S. Yu, UP-Growth: an efficient algorithm for high utility itemset mining, in: Presented at the Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2010, pp. 253–262.
- [16] V.S. Tseng, B.-E. Shie, C.-W. Wu, S.Y. Philip, Efficient algorithms for mining high utility itemsets from transactional databases, IEEE Trans. Knowl. Data Eng. 25 (8) (2012) 1772–1786.
- [17] M. Liu, J. Qu, Mining high utility itemsets without candidate generation, in: Presented at the Proceedings of the 21st ACM International Conference on Information and Knowledge Management, 2012, pp. 55–64.
- [18] P. Fournier-Viger, C.-W. Wu, S. Zida, V.S. Tseng, FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning, in Foundations of Intelligent Systems, vol. 8502, T. Andreassen, H. Christiansen, J.-C. Cubero, Z.W. Raś, Eds., in Lecture Notes in Computer Science, vol. 8502., Cham: Springer International Publishing, 2014, pp. 83–92. doi: 10.1007/978-3-319-08326-1_9.
- [19] S. Zida, P. Fournier-Viger, J.-C.-W. Lin, C.-W. Wu, V.S. Tseng, EFIM: a fast and memory efficient algorithm for high-utility itemset mining, Knowl. Inf. Syst. 51 (2) (2017) 595–625.
- [20] S. Krishnamoorthy, HMiner: efficiently mining high utility itemsets, Expert Syst. Appl. 90 (2017) 168–183.
- [21] J.-F. Qu, M. Liu, P. Fournier-Viger, Efficient algorithms for high utility itemset mining without candidate generation, in High-Utility Pattern Mining, vol. 51, P. Fournier-Viger, J.C.-W. Lin, R. Nkambou, B. Vo, V.S. Tseng, Eds., in Studies in Big Data, vol. 51., Cham: Springer International Publishing, 2019, pp. 131–160. doi: 10.1007/978-3-030-04921-8_5.
- [22] Y. Yan, X. Niu, Z. Zhang, P. Fournier-Viger, L. Ye, F. Min, Efficient high utility itemset mining without the join operation, Inf. Sci. 681 (2024) 121218, <https://doi.org/10.1016/j.ins.2024.121218>.
- [23] T. Truong-Chi, P. Fournier-Viger, A survey of high utility sequential pattern mining, in High-Utility Pattern Mining, vol. 51, P. Fournier-Viger, J.C.-W. Lin, R. Nkambou, B. Vo, V.S. Tseng, Eds., in Studies in Big Data, vol. 51., Cham: Springer International Publishing, 2019, pp. 97–129. doi: 10.1007/978-3-030-04921-8_4.
- [24] X. Liu, G. Chen, W. Zuo, Effective algorithms to mine skyline frequent-utility itemsets, Eng. Appl. Artif. Intel. 116 (2022) 105355, <https://doi.org/10.1016/j.engappai.2022.105355>.
- [25] M.S. Khan, M. Mueyba, F. Coenen, A weighted utility framework for mining association rules, in: Presented at the 2008 Second UKSIM European Symposium on Computer Modeling and simulation, IEEE, 2008, pp. 87–92.

- [26] B. Vo, B. Le, J.J. Jung, A tree-based approach for mining frequent weighted utility itemsets, in: Presented at the International Conference on Computational Collective Intelligence, Springer, 2012, pp. 114–123.
- [27] T. Tran, B. Vo, T.T.N. Le, N.T. Nguyen, Text clustering using frequent weighted utility itemsets, *Cybern. Syst.* 48 (3) (2017) 193–209.
- [28] H. Nguyen, T. Le, M. Nguyen, P. Fournier-Viger, V.S. Tseng, B. Vo, Mining frequent weighted utility itemsets in hierarchical quantitative databases, *Knowl.-Based Syst.* 237 (2022) 107709.
- [29] Z. Deng, Z. Wang, J. Jiang, A new algorithm for fast mining frequent itemsets using N-lists, *Science China Inf. Sci.* 55 (9) (2012) 2008–2030.
- [30] D. Trubold, T. Horváth, Mining strongly closed itemsets from data streams, in *Discovery Science*, vol. 10558, A. Yamamoto, T. Kida, T. Uno, T. Kuboyama, Eds., in *Lecture Notes in Computer Science*, vol. 10558, Cham: Springer International Publishing, 2017, pp. 251–266. doi: 10.1007/978-3-319-67786-6_18.
- [31] M. Zihayat, Y. Chen, A. An, Memory-adaptive high utility sequential pattern mining over data streams, *Mach. Learn.* 106 (6) (2017) 799–836, <https://doi.org/10.1007/s10994-016-5617-1>.
- [32] Y. Baek, et al., Erasable pattern mining based on tree structures with damped window over data streams, *Eng. Appl. Artif. Intel.* 94 (2020) 103735, <https://doi.org/10.1016/j.engappai.2020.103735>.
- [33] H. Nam, U. Yun, E. Yoon, J.-C.-W. Lin, Efficient approach of recent high utility stream pattern mining with indexed list structure and pruning strategy considering arrival times of transactions, *Inf. Sci.* 529 (2020) 1–27.
- [34] H.-F. Li, S.-Y. Lee, Mining frequent itemsets over data streams using efficient window sliding techniques, *Expert Syst. Appl.* 36 (2) (2009) 1466–1477, <https://doi.org/10.1016/j.eswa.2007.11.061>.
- [35] T. Calders, N. Dexters, J.J.M. Gillis, B. Goethals, Mining frequent itemsets in a stream, *Inf. Syst.* 39 (2014) 233–255, <https://doi.org/10.1016/j.is.2012.01.005>.
- [36] W. Xiao, J. Hu, SWEclat: a frequent itemset mining algorithm over streaming data using Spark Streaming, *J. Supercomput.* 76 (10) (2020) 7619–7634, <https://doi.org/10.1007/s11227-020-03190-5>.
- [37] M. Han, H. Cheng, N. Zhang, X. Li, L. Wang, An efficient algorithm for mining closed high utility itemsets over data streams with one dataset scan, *Knowl. Inf. Syst.* 65 (1) (2023) 207–240, <https://doi.org/10.1007/s10115-022-01763-9>.
- [38] M.J. Zaki, Scalable algorithms for association mining, *IEEE Trans. Knowl. Data Eng.* 12 (3) (2000) 372–390.